



Universidad Politécnica
de Madrid

**Escuela Técnica Superior de
Ingenieros Informáticos**



Máster Universitario en Inteligencia Artificial

Trabajo Fin de Máster

**Desarrollo de una Arquitectura Software
para el Entrenamiento de Robots Aéreos
Autónomos Mediante Aprendizaje por
Refuerzo**

Autor: Jordi Florit Ensenyat
Tutor: Martín Molina González

Madrid, Julio - 2026

Este Trabajo Fin de Máster se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Trabajo Fin de Máster

Máster: Máster Universitario en Inteligencia Artificial

Título: Desarrollo de una Arquitectura Software para el Entrenamiento de Robots Aéreos Autónomos Mediante Aprendizaje por Refuerzo

Julio - 2026

Autor: Jordi Florit Ensenyat

Tutor: Martín Molina González

Departamento de Inteligencia Artificial

Escuela Técnica Superior de Ingenieros Informáticos

Universidad Politécnica de Madrid

Índice general

Agradecimientos	1
1. Introducción	4
1.1. Motivación y contexto del proyecto	4
1.2. Objetivos	4
1.3. Estructura del documento	5
2. Fundamentos técnicos y trabajo relacionado	6
2.1. Infraestructura robótica: ROS 2 y Aerostack2	6
2.1.1. ROS 2 como middleware robótico	6
2.1.2. Aerostack2	7
2.1.3. Gymnasium y Stable-Baselines3 en el flujo de aprendizaje	9
2.1.4. Herramientas de visualización y monitorización.	9
2.2. Aprendizaje por refuerzo para el control de UAV	10
2.2.1. PPO como algoritmo de entrenamiento	13
2.3. Simuladores y entornos de entrenamiento para RL en drones	15
2.4. Síntesis y posicionamiento del trabajo	17
3. Diseño de la arquitectura	18
3.1. Arquitectura general	18
3.2. Diseño de la interfaz Gymnasium	19
3.3. Diseño del reinicio de episodios y accionabilidad	21
3.4. Diseño del protocolo de evaluación de políticas	22
4. Implementación de la arquitectura	23
4.1. Organización del software	23
4.2. Integración con Aerostack2 y ROS 2	24
4.3. Herramientas de visualización y monitorización	26
4.4. Implementación de la interfaz Gymnasium	28
4.5. Reinicio determinista del simulador y validación de la accionabilidad	29
4.6. Infraestructura de entrenamiento y vectorización	30
4.7. Verificación técnica de la implementación	30
4.7.1. Pruebas de encapsulamiento y configuración	31
4.7.2. Comprobaciones de integración con Aerostack2	32
4.8. Alcance de la verificación de implementación	33
5. Aplicación y validación de la arquitectura	34
5.1. Propósito y alcance de la aplicación	34
5.2. Formulación de la tarea de control	35

5.3. Recompensa y condiciones de finalización	37
5.4. Curriculum de distancia	37
5.5. Protecciones de frontera	38
5.6. Extensión a ejecución vectorizada	39
5.7. Configuración del entrenamiento PPO	40
5.8. Métricas y criterios de interpretación	41
6. Resultados de la aplicación y validación de la arquitectura	42
6.1. Evolución de la aplicación de entrenamiento	42
6.2. Evidencia de aprendizaje mediante curriculum de distancia	44
6.3. Evaluación de las políticas PPO	45
6.4. Extensión a entrenamiento vectorizado	46
6.5. Hallazgos de integración en el simulador	47
6.6. Limitaciones	48
7. Conclusiones y líneas futuras	49
7.1. Conclusiones	49
7.2. Líneas futuras	50
Bibliografía	52
A. Anexos	55
A.1. Configuraciones representativas	55
A.2. Comandos representativos	56
A.3. Capturas de TensorBoard	57
A.4. Declaración de uso de herramientas de inteligencia artificial	58

Índice de Figuras

2.1. Comunicación entre nodos en ROS mediante <i>topics</i> y servicios. Fuente: Chollet [Chollet, 2021].	7
2.2. Organización por capas de los componentes software de Aerostack2. Fuente: Fernández-Cortizas et al. [Fernández-Cortizas et al., 2023]. . . .	8
2.3. Relación conceptual entre aprendizaje por refuerzo clásico, aprendizaje profundo supervisado y aprendizaje por refuerzo profundo. Fuente: Objectways [Objectways, 2025].	11
2.4. Bucle de interacción entre agente y entorno en aprendizaje por refuerzo. Fuente: AltexSoft [AltexSoft, 2021], basado en la formulación clásica de Sutton y Barto [Sutton y Barto, 2018].	11
2.5. Arquitectura actor-crítico utilizada habitualmente en PPO. Fuente: ResearchGate [ResearchGate, 2024].	14
3.1. Arquitectura general desarrollada, separando la lógica de aprendizaje, la interfaz Gymnasium, la comunicación robótica con Aerostack2 y el simulador con servicio de reinicio.	19
3.2. Ciclo de interacción entre el agente, la interfaz Gymnasium y el simulador durante un episodio.	20
4.1. Integración implementada entre la política de aprendizaje, la interfaz Gymnasium, Aerostack2 y ROS 2.	25
4.2. Visualización en RViz2 del UAV simulado, la trayectoria reciente y el marcador del objetivo durante una ejecución con Aerostack2.	27
4.3. Ciclo implementado de un episodio de aprendizaje por refuerzo sobre AS2TestEnv.	28
5.1. Representación conceptual de la pose de un dron mediante los ángulos de <i>roll</i> , <i>pitch</i> y <i>yaw</i> . Fuente: ResearchGate.	35
5.2. Relación geométrica entre el UAV, el objetivo relativo, la dirección de avance y la orientación de guiñada utilizada en el criterio de <i>path-facing</i>	36
5.3. Extensión vectorizada mediante instancias lógicas asociadas a espacios de nombres diferenciados sobre una infraestructura de simulación compartida.	39
6.1. Tasa de éxito por experimento durante la campaña principal. Cada barra agrega los episodios registrados por el monitor de la ejecución; <i>n</i> indica el número de episodios.	44

6.2. Tasa de éxito en ventana móvil de 50 episodios a lo largo de las tres etapas del curriculum y del entrenamiento vectorizado posterior. Las líneas discontinuas marcan los cambios de etapa, en los que se amplía el rango de distancias muestreado.	45
6.3. Distribución de la distancia final al objetivo en los 100 episodios de la evaluación determinista de Exp016. Las barras azules representan los éxitos y las naranjas los episodios truncados por alcanzar el límite de pasos. La línea discontinua marca el umbral de éxito de 0,4 m: los fallos quedan a su derecha, sin terminaciones de seguridad.	46
A.1. Ejemplo de métricas de <i>rollout</i> de TensorBoard para Exp010. La captura ilustra el formato de consulta de los escalares registrados durante el entrenamiento; la interpretación del experimento se realiza junto con los monitores de episodio y la matriz de resultados.	58

Índice de Tablas

2.1. Relación entre las capas tecnológicas empleadas en el presente trabajo.	9
2.2. Diferencias principales entre aprendizaje profundo supervisado y aprendizaje por refuerzo.	10
3.1. Resumen del contrato principal de la interfaz Gymnasium.	21
3.2. Estados y fallos considerados en el diseño del reinicio de episodios.	22
4.1. Organización principal del software implementado.	24
4.2. Niveles de verificación técnica aplicados a la implementación.	31
4.3. Pruebas de encapsulamiento representativas de la implementación.	32
4.4. Contratos verificados en la integración con Aerostack2.	33
5.1. Componentes de la función de recompensa y condiciones de finalización de la interfaz Gymnasium.	38
5.2. Principales parámetros PPO definidos en la configuración base de entrenamiento.	40
6.1. Ejecuciones relevantes de la aplicación de entrenamiento y evidencia aportada para validar la arquitectura.	43
6.2. Evaluación de las políticas PPO en 100 episodios con semillas fijas. Pasos y tiempo corresponden a la mediana de los episodios con éxito; el error de guiñada es la media absoluta en el instante terminal.	45
A.1. Configuraciones representativas de la fase de estabilización.	56
A.2. Rutas de consulta de configuraciones, scripts y artefactos experimentales en el repositorio.	57
A.3. Resumen del uso de herramientas de inteligencia artificial generativa.	59

Índice de Listings

A.1. Patrón general de entrenamiento PPO a partir de una configuración YAML.	56
A.2. Patrón general de entrenamiento supervisado con reanudación.	57
A.3. Patrón general de evaluación por semillas.	57
A.4. Consulta de métricas TensorBoard para una ejecución concreta.	58

Agradecimientos

Deseo expresar mi más sincero agradecimiento al profesor Martín Molina González, tutor de este trabajo, por su orientación, disponibilidad y acompañamiento durante todo el proceso de elaboración. Asimismo, quiero agradecer a Javier Melero Deza su apoyo técnico, sus valiosas sugerencias y el tiempo dedicado al seguimiento del desarrollo realizado. Su colaboración y disposición para resolver dudas han contribuido de manera significativa a la consecución de los objetivos planteados.

También quiero dar las gracias a mis compañeros y compañeras del máster, con quienes he compartido aprendizajes, retos y experiencias a lo largo de esta etapa. Su compañía ha hecho que mi experiencia viviendo fuera de casa, en Madrid, haya sido mucho más agradable y enriquecedora, tanto en el ámbito académico como en el personal.

A mis amigos y a mi familia, gracias por su apoyo constante, desde la distancia, por su comprensión y por haber estado presentes durante todo este proceso, especialmente en los momentos de mayor esfuerzo y dificultad. Su ánimo y confianza han sido esenciales para poder llegar hasta aquí.

Por último, quiero dedicar un agradecimiento muy especial a Paula, por su paciencia, su apoyo incondicional y su compañía a lo largo de esta etapa. Gracias por escucharme, animarme y ayudarme a mantener la motivación incluso en los momentos más complicados. Su presencia ha sido una parte fundamental de este camino.

Resumen

Este trabajo desarrolla una arquitectura software para el entrenamiento mediante aprendizaje por refuerzo de robots aéreos autónomos en un simulador de multirrotores integrado con Aerostack2 y ROS 2. La arquitectura se valida formulando una tarea de navegación local hacia objetivos relativos con estados iniciales y objetivos variables. Para ello, define observaciones relativas normalizadas, acciones continuas de velocidad lineal y de guiñada, recompensa, terminación, reinicio y monitorización compatibles con Gymnasium.

La contribución principal es la infraestructura que hace operativa esta formulación sobre un sistema robótico simulado. La arquitectura se integra con Aerostack2 y Stable-Baselines3 e incorpora mecanismos para verificar el estado inicial, comprobar la accionabilidad de los comandos, registrar diagnósticos y proteger el vuelo cerca de los límites de seguridad. También permite una ejecución vectorizada acotada con dos drones mediante procesos independientes sobre un simulador compartido.

La validación combina pruebas de encapsulamiento, pruebas de integración con Aerostack2 y entrenamiento PPO en escenarios con estados iniciales y objetivos variables. Las ejecuciones permitieron identificar y mitigar limitaciones del simulador relacionadas con reinicios repetidos y con la persistencia de referencias de velocidad durante operaciones bloqueantes. El entrenamiento se emplea así como una prueba integrada de que la arquitectura genera episodios interpretables, sostiene el ciclo de aprendizaje y se extiende a dos instancias vectorizadas.

Palabras Clave: aprendizaje por refuerzo, vehículos aéreos no tripulados, Aerostack2, ROS 2, Gymnasium, PPO, simulación, validación de arquitecturas software.

Abstract

This thesis develops a software architecture for training autonomous aerial robots through reinforcement learning in a multirotor simulation environment integrated with Aerostack2 and ROS 2. The proposed architecture is validated by formulating a local navigation task in which the robot must reach relative target positions from variable initial states. To achieve this, the Gymnasium interface defines normalized relative observations, continuous linear velocity and yaw rate actions, reward functions, episode termination and reset conditions, and monitoring mechanisms compatible with Gymnasium.

The main contribution of this work is the infrastructure that enables this formulation to operate on a simulated robotic system. The architecture is integrated with Aerostack2 and Stable-Baselines3 and incorporates mechanisms to verify the initial state, ensure the executability of control commands, record diagnostic information, and safeguard flight operations near safety boundaries. It also supports bounded vectorized execution with two drones running as independent processes within a shared simulation.

The validation combines architecture-level encapsulation tests, integration tests with Aerostack2, and PPO training in scenarios with variable initial states and target positions. The experiments made it possible to identify and mitigate simulator limitations related to repeated episode resets and the persistence of velocity references during blocking operations. Consequently, the training process serves as an integrated demonstration that the architecture produces interpretable episodes, sustains the reinforcement learning cycle, and successfully scales to two vectorized simulation instances.

Keywords: reinforcement learning, unmanned aerial vehicles, Aerostack2, ROS 2, Gymnasium, PPO, simulation, software architecture validation.

Capítulo 1

Introducción

1.1. Motivación y contexto del proyecto

Los vehículos aéreos no tripulados han pasado en pocos años de ser una tecnología principalmente experimental a convertirse en una herramienta útil en aplicaciones industriales, logísticas, de inspección y competición. En estos escenarios, la autonomía y la capacidad de respuesta del vehículo son aspectos críticos, especialmente cuando el dron debe seguir referencias espaciales, reaccionar ante cambios del entorno o mantener un comportamiento estable en maniobras rápidas.

El aprendizaje por refuerzo permite abordar parte de estos problemas mediante políticas que se obtienen a partir de la interacción con un entorno. Sin embargo, su aplicación a sistemas robóticos no depende únicamente del algoritmo de aprendizaje, también requiere una formulación consistente de la tarea y una infraestructura que conecte el simulador, el *middleware* robótico y el ciclo de episodios. Si las observaciones, los comandos o los reinicios no representan de forma fiable el comportamiento del vehículo, los datos obtenidos durante el entrenamiento dejan de ser válidos para evaluar una política.

La motivación de este trabajo surge en la intersección entre robótica aérea e inteligencia artificial. En el grupo CVAR (*Computer Vision and Aerial Robotics*) se trabaja con Aerostack2 y ROS 2 como base para el desarrollo y la experimentación con vehículos aéreos. En este contexto, el presente trabajo se centra en construir una arquitectura software para el entrenamiento de robots aéreos autónomos mediante aprendizaje por refuerzo. La aportación se orienta a disponer de una base técnica que permita formular tareas, ejecutar episodios, registrar diagnósticos y validar la interacción entre los componentes antes de extraer conclusiones sobre el aprendizaje.

1.2. Objetivos

El objetivo general de este Trabajo Fin de Máster es desarrollar una arquitectura para el entrenamiento mediante aprendizaje por refuerzo de robots aéreos autónomos en un sistema de simulación integrado con un *stack* robótico.

Para alcanzar este objetivo general, el trabajo se organiza en los siguientes objetivos específicos:

- Analizar el problema de navegación de un robot aéreo en simulación y definir los requisitos funcionales necesarios para la arquitectura y su interfaz de aprendizaje por refuerzo.
- Diseñar e implementar una arquitectura que integre el simulador dinámico con el *middleware* robótico y permita ejecutar el ciclo de interacción entre agente y dron de forma reproducible.
- Validar la arquitectura mediante el entrenamiento de políticas de navegación en escenarios con estados iniciales y objetivos variables, analizando las limitaciones de integración asociadas al reinicio de episodios, la accionabilidad de los comandos y la reproducibilidad de la simulación.

1.3. Estructura del documento

La memoria se organiza como una progresión desde los fundamentos y el diseño de la arquitectura hasta su implementación y validación. Tras esta introducción, el Capítulo 2 presenta los fundamentos técnicos y el trabajo relacionado sobre robótica aérea, aprendizaje por refuerzo y simuladores empleados para el entrenamiento de UAV. El Capítulo 3 define el diseño de la arquitectura, su contrato de interacción y las decisiones de diseño relativas a observaciones, acciones, recompensa, terminación e integración con Aerostack2.

El Capítulo 4 describe la materialización de esta arquitectura en la implementación, incluyendo la conexión con ROS 2, la gestión del ciclo de episodios, los mecanismos de diagnóstico y el soporte para ejecución vectorizada. El Capítulo 5 presenta la aplicación de entrenamiento de una tarea de navegación, empleada para validar de manera integrada la arquitectura construida. El Capítulo 6 expone y analiza los resultados de dicha aplicación, así como las limitaciones de integración identificadas durante la validación. Finalmente, el Capítulo 7 recoge las conclusiones, revisa el cumplimiento de los objetivos y plantea las líneas futuras de trabajo. Los anexos complementan la memoria con la información necesaria para consultar las configuraciones, los registros y los artefactos asociados al proyecto.

Capítulo 2

Fundamentos técnicos y trabajo relacionado

Este capítulo introduce los fundamentos técnicos necesarios para comprender la arquitectura desarrollada y revisa los trabajos relacionados con aprendizaje por refuerzo aplicado a vehículos aéreos no tripulados. No se pretende realizar una revisión exhaustiva de todos los conceptos de robótica o aprendizaje automático, sino seleccionar aquellos elementos que condicionan directamente el diseño posterior como el middleware robótico empleado, la arquitectura aérea sobre la que se ejecuta el sistema, el papel de la simulación y la formulación del problema de aprendizaje.

La organización sigue una progresión de infraestructura a método. En primer lugar, se presentan ROS 2 y Aerostack2 como base software para construir sistemas aéreos autónomos. A continuación, se revisa el aprendizaje por refuerzo profundo en drones y se justifica el uso de PPO como algoritmo de entrenamiento. Después se analizan simuladores y entornos de entrenamiento relevantes, con especial atención a las implicaciones de usar un simulador integrado en un *stack* robótico. Finalmente, se sintetiza el posicionamiento del presente trabajo respecto a la literatura revisada.

2.1. Infraestructura robótica: ROS 2 y Aerostack2

El desarrollo de sistemas aéreos autónomos no consiste únicamente en programar un algoritmo de control. Un UAV operativo requiere coordinar estimación de estado, control, comunicaciones, gestión de modos de vuelo, seguridad, planificación y, en muchos casos, percepción. Esta integración es uno de los principales motivos por los que los trabajos de robótica suelen apoyarse en arquitecturas software reutilizables que permiten que los algoritmos se conecten con un sistema completo en lugar de ejecutarse como componentes aislados.

2.1.1. ROS 2 como middleware robótico

ROS 2 proporciona una infraestructura de comunicación para sistemas robóticos distribuidos. Un sistema ROS 2 se organiza como un grafo de nodos que intercambian información mediante distintos mecanismos de comunicación. Los *topics* permiten publicar flujos continuos de datos, los servicios modelan interacciones de petición y respuesta, y las acciones representan tareas de mayor duración con realimentación

y posibilidad de cancelación [Open Robotics, 2026]. Esta separación resulta especialmente útil en robótica aérea, donde conviven señales continuas de estado con órdenes discretas de ciclo de vida, como armar, despegar o aterrizar.

La Figura 2.1 muestra esta idea de forma conceptual. Un nodo puede actuar como publicador o suscriptor de un *topic*, lo que permite distribuir información de estado o medidas de sensores. El mismo nodo también puede ofrecer o consumir servicios, adecuados para operaciones puntuales que requieren una respuesta explícita. Esta distinción es importante en el presente trabajo porque el ciclo de aprendizaje combina ambos tipos de interacción: lectura recurrente del estado del UAV y ejecución de órdenes de preparación o control sobre la plataforma simulada.

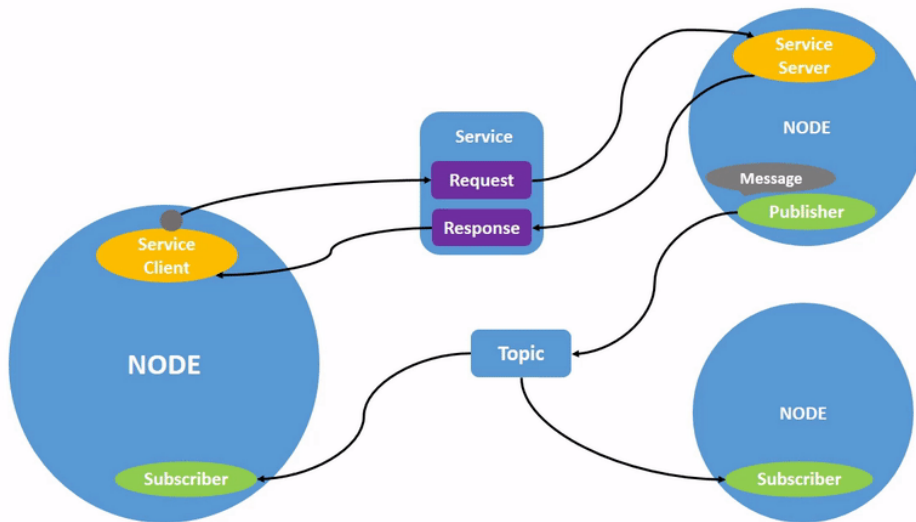


Figura 2.1: Comunicación entre nodos en ROS mediante *topics* y servicios. Fuente: Chollet [Chollet, 2021].

Desde el punto de vista de este trabajo, ROS 2 se usa como la capa que permite comunicar la interfaz de aprendizaje de la arquitectura con el sistema robótico simulado. La política de aprendizaje por refuerzo no publica directamente en todos los *topics* ni gestiona todos los servicios disponibles; esa responsabilidad queda encapsulada en las capas de Aerostack2 y en la interfaz desarrollada. Esta decisión evita que el algoritmo de aprendizaje dependa de detalles de comunicación de bajo nivel.

2.1.2. Aerostack2

Sobre esta base se sitúa Aerostack2. Inicialmente Aerostack fue propuesto como un marco software abierto, modular y multicapa para el desarrollo de sistemas aéreos autónomos complejos [Sánchez-López et al., 2017]. Su objetivo era proporcionar una arquitectura reutilizable capaz de integrar componentes de control, percepción, planificación y ejecución de misiones en robótica aérea. Esta idea es relevante porque desplaza el foco desde el algoritmo aislado hacia el sistema completo que permite ejecutar una misión aérea.

Por otra parte, Aerostack2 constituye la evolución de esta línea sobre ROS 2. Su diseño enfatiza la independencia de plataforma, la arquitectura basada en *plugins*,

2.1. Infraestructura robótica: ROS 2 y Aerostack2

el soporte para sistemas multirrobot y la definición de misiones mediante comportamientos reutilizables [Fernández-Cortizas et al., 2023]. En lugar de imponer un único simulador o una única plataforma física, Aerostack2 actúa como capa de abstracción para conectar distintos vehículos, simuladores y componentes de misión. Esta característica lo hace adecuado para trabajos en los que interesa aproximar la experimentación en simulación a un flujo de robótica aérea operativo.

La arquitectura de Aerostack2 puede entenderse como una organización por capas, resumida en la Figura 2.2. En los niveles inferiores se sitúan el middleware, los mensajes de comunicación y las interfaces con plataformas o sensores. Sobre ellos se construyen funciones robóticas básicas, como estimación de estado, control de movimiento o gestión de emergencias. A un nivel superior aparecen los comportamientos de robot, como despegue, aterrizaje, seguimiento de trayectorias o detección de marcadores. Finalmente, las capas de misión y aplicación permiten componer esas capacidades para casos de uso concretos.

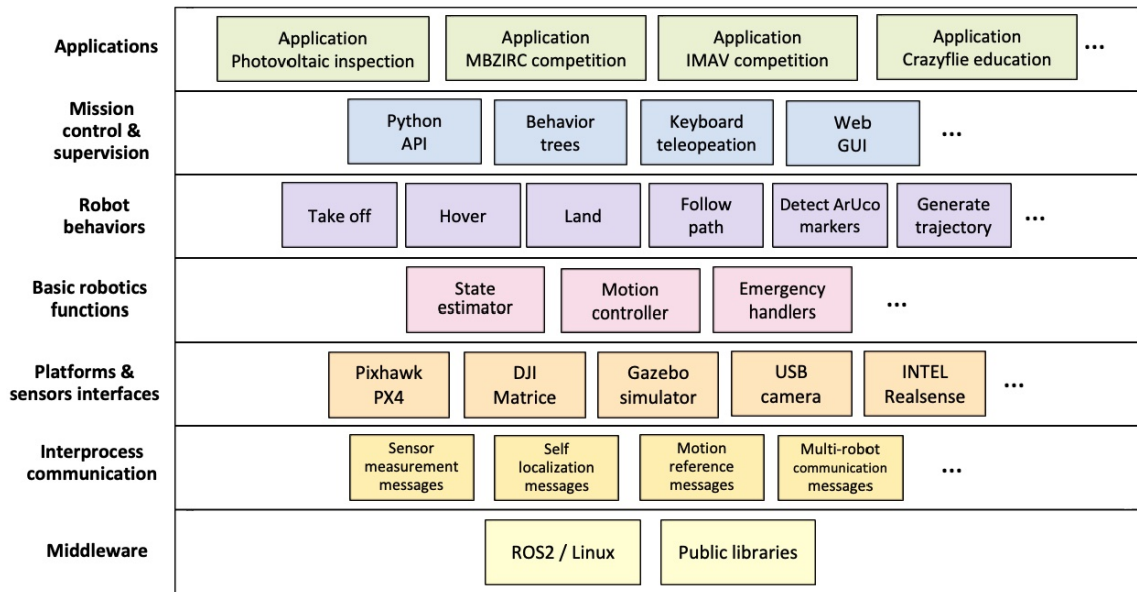


Figura 2.2: Organización por capas de los componentes software de Aerostack2. Fuente: Fernández-Cortizas et al. [Fernández-Cortizas et al., 2023].

Esta organización por capas ayuda a explicar por qué Aerostack2 resulta especialmente útil para el presente trabajo. La interfaz de aprendizaje por refuerzo de la arquitectura no sustituye a los controladores, estimadores o comportamientos de Aerostack2, sino que se conecta por encima de ellos para emitir consignas de velocidad dentro de una infraestructura robótica ya existente. Así, el problema de aprendizaje se mantiene centrado en decidir acciones de alto nivel, mientras que la ejecución robótica queda delegada en componentes especializados del *stack*.

Nivel	Función principal	Papel en este trabajo
ROS 2	Comunicación entre nodos mediante <i>topics</i> , servicios y acciones.	Transporte de órdenes, estados y operaciones de ciclo de vida.
Aerostack2	Arquitectura aérea sobre ROS 2 con interfaces y comportamientos reutilizables.	Sustrato robótico usado por la arquitectura de entrenamiento.
Simulador AS2	Ejecución simulada del multirrotor y de sus interfaces robóticas.	Fuente de experiencia para entrenamiento y validación funcional.
Gymnasium/SB3	Interfaz estándar de aprendizaje por refuerzo y entrenamiento PPO.	Capa de interacción del agente con observaciones, acciones y recompensas.

Cuadro 2.1: Relación entre las capas tecnológicas empleadas en el presente trabajo.

El Cuadro 2.1 resume la función de cada capa. Esta separación ayuda a delimitar la contribución del trabajo. No se propone un nuevo middleware robótico ni una nueva arquitectura aérea, sino una arquitectura software reproducible que integra una interfaz Gymnasium con una plataforma aérea basada en ROS 2 y Aerostack2. Por ello, los capítulos posteriores distinguen entre la infraestructura robótica reutilizada, el diseño del problema de aprendizaje y la implementación de la interfaz Gymnasium.

2.1.3. Gymnasium y Stable-Baselines3 en el flujo de aprendizaje

Gymnasium proporciona una API común para definir entornos de aprendizaje por refuerzo mediante operaciones como `reset` y `step`, junto con espacios de observación y acción explícitos [Towers et al., 2024; Farama Foundation, 2026]. Su papel en este trabajo no es simular por sí mismo la dinámica del UAV, sino ofrecer el contrato mediante el cual un algoritmo de aprendizaje puede interactuar de forma uniforme con una tarea implementada sobre ROS 2 y Aerostack2. Esta separación permite que la lógica del agente no dependa directamente de los detalles de comunicación robótica.

Stable-Baselines3 aporta implementaciones mantenidas de algoritmos de aprendizaje por refuerzo profundo y utilidades de entrenamiento compatibles con este tipo de entornos, incluida la implementación de PPO utilizada más adelante [Stable-Baselines3 Contributors, 2026]. En el presente trabajo se emplea como capa de entrenamiento y monitorización, mientras que la definición del problema, la recompensa, los reinicios y las condiciones de seguridad permanecen en la interfaz desarrollada dentro de la arquitectura. Por tanto, Gymnasium estandariza la interfaz y Stable-Baselines3 proporciona el algoritmo, pero la validez experimental depende de que la adaptación robótica produzca transiciones físicas fiables.

2.1.4. Herramientas de visualización y monitorización.

En una infraestructura de aprendizaje por refuerzo integrada con ROS 2, la observación del sistema no se limita a las métricas internas del algoritmo. RViz2 proporciona una visualización espacial del estado del UAV, de sus referencias y de los marcadores publicados en el grafo ROS 2, por lo que resulta útil para validar cualitativamente que la escena simulada y los comandos robóticos son coherentes. TensorBoard,

por su parte, permite inspeccionar las métricas emitidas por Stable-Baselines3 y por los monitores de episodio, como recompensa media, longitud de episodio y señales auxiliares registradas durante la ejecución. En este trabajo, ambas herramientas se emplean como soporte de validación y diagnóstico, no como sustituto de las métricas cuantitativas de la arquitectura.

2.2. Aprendizaje por refuerzo para el control de UAV

El aprendizaje por refuerzo se diferencia de otros enfoques de aprendizaje automático porque el conocimiento no se obtiene a partir de un conjunto fijo de pares entrada-salida, sino mediante interacción secuencial con un entorno. En aprendizaje supervisado clásico, una red neuronal aprende a aproximar una función usando ejemplos etiquetados; en aprendizaje por refuerzo, en cambio, el agente debe actuar, observar las consecuencias de sus acciones y ajustar su comportamiento según una señal de recompensa. Esta diferencia es esencial en robótica: un robot no solo debe reconocer patrones, sino decidir acciones que modifican el estado físico del sistema.

Esta diferencia puede resumirse en el Cuadro 2.2. En aprendizaje profundo supervisado, el objetivo habitual es aprender una relación estática entre entradas y etiquetas. En aprendizaje por refuerzo, el problema es dinámico las decisiones modifican los datos que se observarán después. Por ello, la calidad del aprendizaje no depende solo del modelo neuronal, sino también de la definición del entorno, de la recompensa y de la estrategia de exploración [Sutton y Barto, 2018; Molina, 2025b].

Aspecto	Aprendizaje profundo supervisado	Aprendizaje por refuerzo
Datos de entrenamiento	Conjunto de ejemplos normalmente fijo y etiquetado.	Experiencia generada por interacción con el entorno.
Señal de aprendizaje	Error entre predicción y etiqueta conocida.	Recompensa escalar y acumulativa.
Salida del modelo	Predicción, clasificación o regresión.	Acción o distribución de acciones.
Dependencia temporal	Las muestras suelen tratarse como ejemplos independientes.	Cada acción condiciona los estados y recompensas futuros.
Relevancia en robótica	Percepción, estimación o reconocimiento.	Control, navegación y toma de decisiones secuencial.

Cuadro 2.2: Diferencias principales entre aprendizaje profundo supervisado y aprendizaje por refuerzo.

La Figura 2.3 complementa esta comparación. El aprendizaje por refuerzo clásico puede representarse mediante tablas de valor cuando los estados y acciones son discretos y de tamaño limitado. El aprendizaje profundo supervisado, por su parte, utiliza redes neuronales para extraer patrones a partir de datos etiquetados. El aprendizaje por refuerzo profundo combina ambas ideas: emplea redes neuronales para aproximar políticas, funciones de valor o ambas, pero mantiene el ciclo de interacción con el entorno propio del aprendizaje por refuerzo. Esta combinación es la que permite aplicar RL a problemas con estados continuos, observaciones de alta dimensión o espacios de acción complejos, como los que aparecen en robótica aérea.

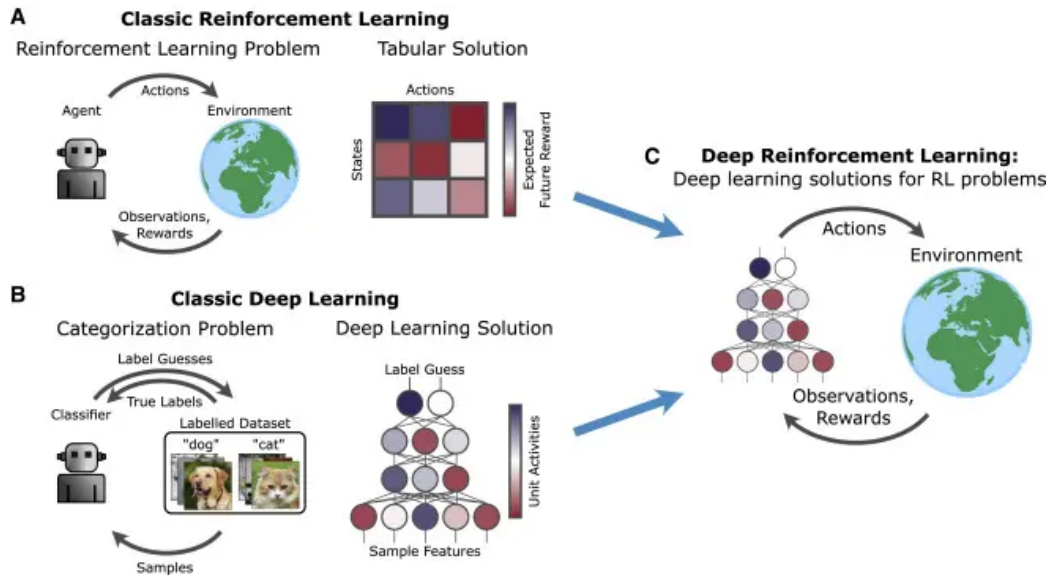


Figura 2.3: Relación conceptual entre aprendizaje por refuerzo clásico, aprendizaje profundo supervisado y aprendizaje por refuerzo profundo. Fuente: Objectways [Objectways, 2025].

La Figura 2.4 resume el ciclo básico de aprendizaje por refuerzo. En cada instante t , el agente recibe un estado u observación S_t , selecciona una acción A_t y el entorno responde con una nueva situación S_{t+1} y una recompensa R_{t+1} [Sutton y Barto, 2018]. En un problema robótico, el entorno incluye tanto la dinámica del vehículo como la infraestructura que ejecuta las órdenes y proporciona medidas del estado. En este trabajo, ese papel lo desempeñan conjuntamente Aerostack2, ROS 2 y el simulador multirrotor.

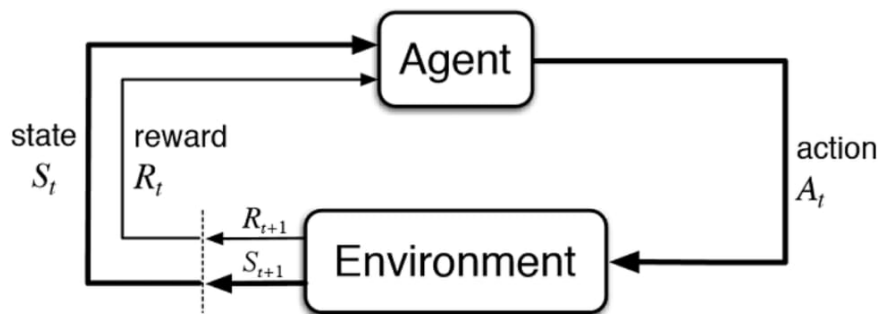


Figura 2.4: Bucle de interacción entre agente y entorno en aprendizaje por refuerzo. Fuente: AltexSoft [AltexSoft, 2021], basado en la formulación clásica de Sutton y Barto [Sutton y Barto, 2018].

Formalmente, un problema de aprendizaje por refuerzo suele expresarse como un proceso de decisión de Markov. Esta formulación se define mediante un conjunto de estados S , un conjunto de acciones A , una función de transición P , una función de recompensa R y un factor de descuento $\gamma \in [0, 1]$ [Sutton y Barto, 2018]. La política π determina cómo actúa el agente: puede representarse como una función determinista $\pi(s) = a$ o como una distribución de probabilidad $\pi(a|s)$ sobre acciones posibles. El objetivo del aprendizaje consiste en encontrar una política que maximice la recompensa acumulada esperada.

El retorno descontado desde un instante t se define como:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}.$$

Este valor resume la utilidad futura de una secuencia de decisiones. El factor de descuento γ permite ponderar las recompensas lejanas: valores cercanos a uno consideran horizontes más largos, mientras que valores menores favorecen recompensas inmediatas. En robótica aérea esta distinción es importante, porque una acción puede parecer buena a corto plazo, por ejemplo aumentar la velocidad hacia el objetivo, pero generar trayectorias inestables o salidas de la región segura en pasos posteriores.

A partir del retorno se definen dos funciones habituales. La función de valor $V^\pi(s)$ estima el retorno esperado al seguir la política π desde un estado s . La función de valor-acción $Q^\pi(s, a)$ estima el retorno esperado al ejecutar una acción concreta a en el estado s y continuar después con la política π . La diferencia entre ambas magnitudes permite definir la ventaja:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

La ventaja mide si una acción concreta es mejor o peor que el comportamiento medio esperado en ese estado. Esta idea será relevante para PPO, ya que el algoritmo usa estimaciones de ventaja para decidir en qué dirección debe modificarse la política.

En el contexto de vehículos aéreos no tripulados, el aprendizaje por refuerzo resulta atractivo porque permite optimizar comportamientos difíciles de programar mediante reglas cerradas. Las revisiones recientes muestran aplicaciones en planificación, navegación y control de UAV, especialmente en escenarios con dinámica no lineal, espacios de acción continuos o transferencia desde simulación [Azar et al., 2021; Chan et al., 2024]. Sin embargo, estas mismas revisiones subrayan limitaciones importantes: necesidad de muchas interacciones, sensibilidad al diseño de la recompensa, dependencia de la fidelidad del simulador y dificultad de trasladar políticas entrenadas a plataformas reales.

Una parte significativa de los trabajos se ha centrado en control a bajo nivel, es decir, en aprender políticas que actúan cerca de los actuadores o de las consignas de actitud. Pi et al. propusieron un controlador de bajo nivel basado en aprendizaje por refuerzo capaz de generalizar a vehículos multirrotor con distintas configuraciones físicas [Pi et al., 2021]. En una línea relacionada, Yoo et al. combinaron controladores deterministas clásicos con una política aprendida, mostrando que una aproximación híbrida puede mejorar la convergencia y el seguimiento de trayectorias mediante control en velocidad [Yoo et al., 2021]. Estos trabajos son relevantes para una posible extensión jerárquica del presente trabajo, aunque la arquitectura desarrollada se sitúa actualmente en un nivel de control más alto: la política genera comandos de velocidad lineal y velocidad angular en yaw, no señales de motor ni tasas angulares completas.

Otra línea activa es la del vuelo agresivo y el empuje del dron hacia sus límites dinámicos. Kaufmann et al. demostraron en *Deep Drone Acrobatics* que es posible entrenar en simulación una política para realizar maniobras acrobáticas y transferirla al dron real sin ajuste fino posterior [Kaufmann et al., 2020]. Más adelante, los mismos autores analizaron comparativamente distintas abstracciones de control y observaron que

las políticas que actúan sobre *body-rates* y *thrust* resultan más robustas en transferencia que aquellas que operan directamente sobre los motores [Kaufmann et al., 2022]. Este punto muestra que el nivel de abstracción de la acción no es una decisión secundaria, sino un elemento central del diseño del problema de aprendizaje.

Los resultados recientes en vuelo agresivo refuerzan esta idea. Eschmann et al. mostraron que, con una arquitectura adecuada y un simulador muy optimizado, es posible entrenar controladores de bajo nivel en tiempos extraordinariamente reducidos y mantener capacidad de transferencia a plataformas reales [Eschmann et al., 2024]. En paralelo, Song et al. estudiaron de manera sistemática la competición entre control óptimo y aprendizaje por refuerzo en un problema de carrera autónoma con drones, concluyendo que el aprendizaje por refuerzo puede superar al control óptimo cuando el objetivo se formula directamente a nivel de tarea y se entrena con suficiente robustez frente a incertidumbres del modelo [Song et al., 2023]. Este resultado es pertinente para el contexto de A2RL que motiva el trabajo, aunque debe interpretarse como antecedente de la literatura y no como evidencia del rendimiento de la arquitectura aquí desarrollada.

La literatura revisada revela, por tanto, una diversidad de formulaciones. Muchos trabajos se centran en estabilización de actitud, control a nivel de actuadores, vuelo acrobático o navegación *end-to-end* basada en percepción [Kaufmann et al., 2020; Pi et al., 2021; Yoo et al., 2021]. A partir de esas obras puede inferirse que existe menos atención a controladores intermedios integrados en un *stack* robótico de ROS 2, donde la acción se expresa como $[v_x, v_y, v_z, v_{yaw}]$ y la observación principal como un estado relativo normalizado $[dx, dy, dz, d_{yaw}]$. Esta inferencia ayuda a delimitar el espacio de contribución del presente trabajo sin convertirlo en una afirmación absoluta sobre toda la literatura existente.

2.2.1. PPO como algoritmo de entrenamiento

PPO, *Proximal Policy Optimization*, pertenece a la familia de métodos de gradiente de política. Estos métodos no aprenden únicamente una función que valora acciones, sino que optimizan directamente una política parametrizada $\pi_\theta(a|s)$ [Schulman et al., 2017]. En aprendizaje por refuerzo profundo, esta política se implementa habitualmente mediante una red neuronal que recibe una observación y produce una distribución de acciones. En espacios continuos, como el empleado en este trabajo, la acción puede interpretarse como una muestra o una salida de una distribución sobre velocidades admisibles.

PPO suele implementarse con una arquitectura actor-crítico. El actor representa la política y decide qué acción ejecutar. El crítico estima el valor de los estados o de las acciones y proporciona una señal auxiliar para evaluar si la acción tomada fue mejor o peor de lo esperado. La Figura 2.5 muestra este esquema: las trayectorias generadas por la interacción con el entorno se usan para actualizar tanto el crítico como la política del actor. Esta separación es útil porque el actor aprende a decidir, mientras que el crítico reduce la varianza de la estimación del gradiente mediante información de valor.

2.2. Aprendizaje por refuerzo para el control de UAV

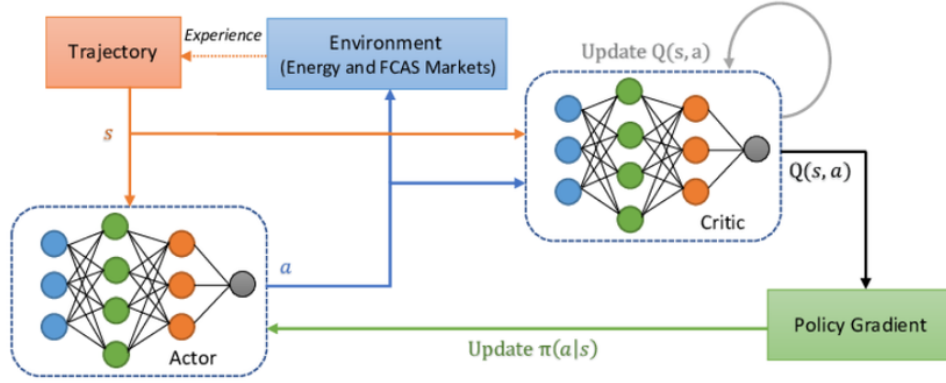


Figura 2.5: Arquitectura actor-crítico utilizada habitualmente en PPO. Fuente: ResearchGate [ResearchGate, 2024].

En una arquitectura actor-crítico, el crítico permite estimar la ventaja sin esperar siempre al retorno completo de un episodio. Una aproximación habitual es partir del error temporal:

$$\delta_t = R_{t+1} + \gamma V(s_{t+1}) - V(s_t),$$

que compara la recompensa inmediata más el valor estimado del estado siguiente con el valor estimado del estado actual. En implementaciones modernas de PPO se emplea frecuentemente una estimación de ventaja generalizada, que combina varios errores temporales ponderados:

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}.$$

El parámetro λ controla el compromiso entre sesgo y varianza: valores altos incorporan más información de retornos largos, mientras que valores bajos se apoyan más en el valor estimado por el crítico. Esta explicación es relevante para interpretar parámetros como `gamma` y `gae_lambda` en la configuración de entrenamiento del presente trabajo [Molina, 2025a; Molina, 2025c].

La dificultad principal de los métodos de gradiente de política es que una actualización demasiado grande puede destruir una política que estaba funcionando razonablemente bien. PPO aborda este problema comparando la política nueva con la política anterior mediante la razón de probabilidades:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}.$$

Si esta razón se aleja demasiado de uno, significa que la nueva política asigna una probabilidad muy distinta a la acción tomada respecto a la política que generó los datos. PPO limita ese cambio mediante una función objetivo recortada:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

El parámetro ϵ define cuánto puede separarse la nueva política de la anterior en una actualización. Si la ventaja estimada $\hat{A}_t$ es positiva, la actualización aumenta la probabilidad de la acción, pero solo hasta el límite permitido por el recorte. Si la ventaja

es negativa, reduce esa probabilidad, de nuevo sin permitir cambios excesivos. Este mecanismo proporciona una forma práctica de mantener las actualizaciones cerca de la política que generó las muestras, evitando saltos demasiado bruscos [Schulman et al., 2017].

El funcionamiento práctico de PPO puede resumirse como un ciclo de cinco pasos. Primero, la política actual interactúa con el entorno y genera trayectorias de estados, acciones, recompensas y finales de episodio. Segundo, el crítico estima valores y se calculan ventajas para cada transición. Tercero, se actualiza el actor maximizando la función objetivo recortada. Cuarto, se entrena el crítico para reducir el error entre sus valores estimados y los retornos objetivo. Quinto, se puede añadir un término de entropía para favorecer exploración y evitar que la política se vuelva determinista demasiado pronto. En la práctica, muchas implementaciones optimizan una combinación de pérdida de política, pérdida de valor y bonificación de entropía [Schulman et al., 2017; Stable-Baselines3 Contributors, 2026]:

$$L(\theta) = L^{CLIP}(\theta) - c_1 L^{VF}(\theta) + c_2 S[\pi_\theta].$$

Aquí, L^{VF} representa el error de la función de valor, $S[\pi_\theta]$ la entropía de la política, y c_1 y c_2 ponderan la importancia relativa de ambos términos. Esta expresión permite entender por qué la configuración de PPO incluye parámetros específicos para la función de valor, la exploración y la norma del gradiente, no se entrena solo una red que “elige acciones”, sino un sistema de optimización que equilibra mejora de política, estimación de valor y estabilidad.

En el presente trabajo, PPO resulta adecuado por tres motivos. En primer lugar, el espacio de acciones es continuo y acotado, ya que la política genera velocidades lineales y velocidad de guiñada. En segundo lugar, su implementación en Stable-Baselines3 se integra de forma directa con entornos Gymnasium y entornos vectorizados. En tercer lugar, el mecanismo de *clipping* reduce el riesgo de actualizaciones inestables, un aspecto relevante cuando cada muestra requiere interactuar con un simulador robótico [Stable-Baselines3 Contributors, 2026]. La elección de PPO debe entenderse, por tanto, como una decisión de diseño para construir un pipeline experimental estable, no como evidencia previa de que la política aprendida supere a un controlador clásico.

Desde una perspectiva robótica, PPO también impone una restricción metodológica: la calidad del aprendizaje depende de la definición del entorno. Observaciones mal normalizadas, acciones con escalas inadecuadas, recompensas ambiguas o reinicios poco fiables pueden producir políticas aparentemente entrenadas pero incapaces de resolver la tarea. Por ello, los capítulos posteriores no se limitan a indicar que se usa PPO, sino que describen explícitamente el contrato Gymnasium, la formulación de observaciones y acciones, la recompensa, las condiciones de finalización y la infraestructura de validación.

2.3. Simuladores y entornos de entrenamiento para RL en drones

El entrenamiento mediante aprendizaje por refuerzo en robótica aérea depende fuertemente del simulador empleado. Un simulador adecuado debe ofrecer fidelidad dinámica suficiente para capturar el comportamiento del vehículo, pero también un coste

2.3. Simuladores y entornos de entrenamiento para RL en drones

computacional reducido que permita generar experiencia de entrenamiento [Chan et al., 2024]. La literatura reciente coincide en que este equilibrio entre realismo, velocidad y facilidad de integración es uno de los factores determinantes del éxito de una solución basada en aprendizaje por refuerzo [Azar et al., 2021; Chan et al., 2024].

Entre los entornos influyentes se encuentra Flightmare, un simulador de cuadricópteros diseñado para desacoplar el renderizado y la física, lo que permite combinar flexibilidad con una velocidad de simulación elevada [Song et al., 2021]. Su interés para aprendizaje por refuerzo reside en que puede simular centenares de drones en paralelo, lo cual lo convierte en una referencia cuando el objetivo es maximizar el rendimiento de muestreo. Desde otra perspectiva, Air Learning amplió el uso de AirSim con un entorno tipo Gym para navegación autónoma, introduciendo aleatorización del dominio, *hardware-in-the-loop* y métricas de “calidad de vuelo” orientadas al despliegue en plataformas con recursos limitados [Krishnan et al., 2021]. Este trabajo muestra que el valor de un simulador para aprendizaje por refuerzo no depende solo de la dinámica, sino también de su capacidad para aproximar las restricciones computacionales del sistema final.

En una dirección más ligera y abierta, Panerati et al. propusieron un entorno Gym basado en PyBullet para cuadricópteros individuales y multiagente, explícitamente orientado a facilitar el estudio de enfoques de control y de aprendizaje [Panerati et al., 2021]. Su aportación es especialmente interesante para trabajos académicos porque reduce la fricción de entrada, ofrece portabilidad y proporciona ejemplos de control convencional y de tareas de aprendizaje por refuerzo. Finalmente, *Learning to Fly in Seconds* plantea un simulador extremadamente optimizado para entrenamiento de controladores de bajo nivel, subrayando que la eficiencia computacional puede llegar a ser tan importante como la fidelidad dinámica cuando el objetivo es acelerar el ciclo de investigación [Eschmann et al., 2024].

Las revisiones de conjunto confirman esta diversidad de soluciones. Chan et al. comparan simuladores orientados a aprendizaje por refuerzo desde la perspectiva de sus prestaciones, interfaces y capacidades de integración, y concluyen que no existe una herramienta universalmente superior, sino distintas elecciones dependiendo del problema, del tipo de aprendizaje y del nivel de realismo requerido [Chan et al., 2024]. Desde la perspectiva de este trabajo, esta conclusión es importante: el simulador de multirrotores de Aerostack2 puede no ofrecer el fotorrealismo de otras plataformas, pero su ejecución sin interfaz gráfica y su integración natural con ROS 2 lo hacen atractivo para construir una infraestructura de entrenamiento conectada con un *stack* robótico real.

En este punto conviene precisar el alcance de la vectorización. El soporte para varios drones en un único simulador compartido y un único proceso Python permite validar la viabilidad funcional de escenarios con N instancias del entorno. En la infraestructura de entrenamiento preparada, la opción prudente es utilizar vectorización síncrona, por ejemplo mediante `DummyVecEnv`, antes de plantear esquemas asíncronos o afirmaciones de aceleración. Sin métricas de rendimiento comparables, la vectorización debe presentarse como capacidad de la infraestructura, no como mejora cuantificada del entrenamiento.

2.4. Síntesis y posicionamiento del trabajo

La revisión anterior muestra que el aprendizaje por refuerzo se ha aplicado a problemas diversos de robótica aérea, desde la estabilización hasta la navegación y el vuelo de altas prestaciones [Kaufmann et al., 2020; Pi et al., 2021; Yoo et al., 2021; Song et al., 2023]. No obstante, la utilidad de estos métodos en un sistema robótico depende de que el entorno de entrenamiento represente de manera consistente el ciclo de interacción con el vehículo. La abstracción de las acciones, la definición de las observaciones y la calidad del simulador condicionan la validez de las trayectorias con las que se entrena y evalúa una política [Kaufmann et al., 2022; Chan et al., 2024].

La literatura también evidencia que la integración entre un entorno de aprendizaje y un *stack* robótico introduce requisitos que no quedan resueltos al disponer únicamente de un simulador o de una interfaz Gym independiente. En particular, deben garantizarse la inicialización de los episodios, la recepción efectiva de los comandos, la comunicación con el *middleware* y el registro de información suficiente para diagnosticar el comportamiento observado. Estas cuestiones resultan especialmente relevantes en entornos basados en ROS 2 y Aerostack2, donde el agente interactúa con una arquitectura robótica distribuida [Fernández-Cortizas et al., 2023; Song et al., 2021; Krishnan et al., 2021; Panerati et al., 2021].

En este contexto se sitúa el presente Trabajo Fin de Máster. Su contribución principal consiste en el diseño e implementación de una arquitectura software reproducible para el entrenamiento mediante aprendizaje por refuerzo de robots aéreos autónomos, integrada con el simulador y con Aerostack2. La arquitectura incorpora una interfaz Gymnasium que formula una tarea de navegación mediante observaciones relativas al objetivo y comandos continuos de velocidad, y proporciona los mecanismos necesarios para reiniciar episodios, aplicar acciones, establecer condiciones de terminación y registrar diagnósticos de la ejecución.

El entrenamiento de una política PPO se emplea como una validación integrada de esta arquitectura. Su finalidad es comprobar que el ciclo completo entre agente, interfaz Gymnasium, simulador y *middleware* permite generar episodios interpretables y sostener el aprendizaje en escenarios con estados iniciales y objetivos variables. Asimismo, la ejecución vectorizada se incorpora como una capacidad de la arquitectura cuya viabilidad se verifica dentro de las limitaciones de hardware disponibles. De este modo, el trabajo prioriza la construcción y la validación técnica de la arquitectura, mientras que los resultados de entrenamiento se interpretan como evidencia de que dicha infraestructura cumple su propósito.

Capítulo 3

Diseño de la arquitectura

Este capítulo presenta el diseño de la arquitectura desarrollada para integrar una interfaz de aprendizaje por refuerzo orientada al control de un UAV en Aerostack2. La descripción se centra en la arquitectura y en las decisiones que permiten integrar Gymnasium con la interfaz robótica de ROS 2. No se aborda aquí la programación detallada de los módulos, sino las decisiones de diseño que estructuran la solución y delimitan su comportamiento.

3.1. Arquitectura general

La arquitectura se organiza como una cadena de responsabilidades que separa el entrenamiento del agente, la interfaz Gymnasium y la comunicación con el simulador. En el nivel superior se sitúan los scripts de entrenamiento, encargados de instanciar el entorno y conectar el algoritmo de aprendizaje por refuerzo con la API estándar de Gymnasium. Esta API actúa como frontera entre el agente y el sistema robótico, de modo que el agente no opera directamente sobre ROS 2, sino sobre un contrato de observaciones, acciones, recompensas y condiciones de finalización.

El componente central de esta frontera es la clase `AS2TestEnv`, definida como una especialización de `gym.Env` y registrada como `AS2TestEnv-v0`. Su función de diseño consiste en traducir cada paso del contrato Gymnasium en órdenes comprensibles por Aerostack2 y, a la inversa, convertir el estado relativo del UAV en una observación apta para el agente. La comunicación con Aerostack2 se realiza mediante `as2_python_api.DroneInterface`, mientras que el envío de velocidades se encapsula a través de `SpeedMotion` y del método `send_speed_command_with_yaw_speed`. De este modo, el agente queda desacoplado de los detalles de bajo nivel de ROS 2 y del simulador multirrotor de Aerostack2 [Fernández-Cortizas et al., 2023].

La Figura 3.1 resume esta separación por capas. La interacción avanza desde los scripts de entrenamiento hacia la API Gymnasium, la clase `AS2TestEnv`, la interfaz `DroneInterface/SpeedMotion`, ROS 2 y, finalmente, el AS2 Multirrotor Simulator. La última capa incorpora, en la versión final de la arquitectura, un overlay con un servicio explícito de reinicio de estado. Esta estructura sigue un criterio de aislamiento. El entrenamiento decide acciones en un espacio matemático acotado, la interfaz Gymnasium aplica las transformaciones necesarias y Aerostack2 ejecuta los comandos en el simulador. Además, la inicialización de ROS 2 se diseña como un recurso único

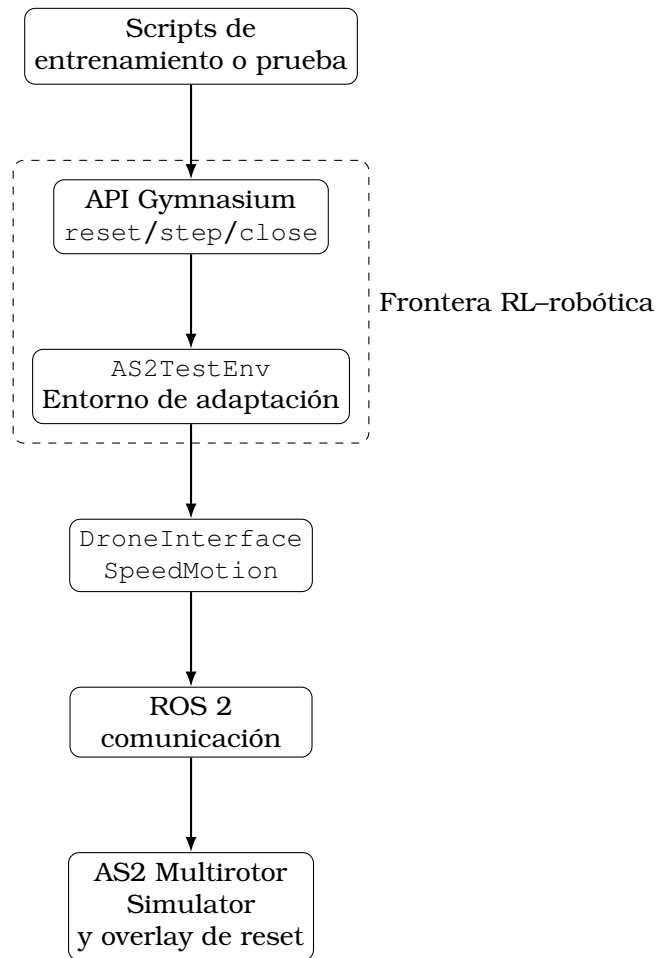


Figura 3.1: Arquitectura general desarrollada, separando la lógica de aprendizaje, la interfaz Gymnasium, la comunicación robótica con Aerostack2 y el simulador con servicio de reinicio.

por proceso mediante las variables de control `_rcipy_initialized` y `_rcipy_lock`, con el fin de evitar inicializaciones repetidas y reducir riesgos de carrera cuando se crean varias instancias del entorno.

La incorporación del servicio de reinicio supone una parte crítica del diseño. El reinicio deja de depender únicamente de comandos de velocidad o de comportamientos de navegación de Aerostack2, y pasa a contar con una operación propia del simulador para restaurar pose, velocidades y referencias internas. Esta decisión se introduce porque, en aprendizaje por refuerzo, un episodio solo es válido si el estado inicial y la accionabilidad posterior al reset son verificables. El entorno decide cuándo reiniciar, el simulador aplica la mutación física del estado y el propio entorno vuelve a certificar telemetría y movimiento antes de entregar la siguiente transición a PPO.

3.2. Diseño de la interfaz Gymnasium

La interfaz Gymnasium se diseña como una adaptación entre el ciclo estándar `reset-step-close` y las operaciones necesarias para preparar, gobernar y finalizar el vuelo simulado. Esta decisión permite que el agente interactúe con el UAV mediante una interfaz conocida en aprendizaje por refuerzo, mientras que la lógica específica de

3.2. Diseño de la interfaz Gymnasium

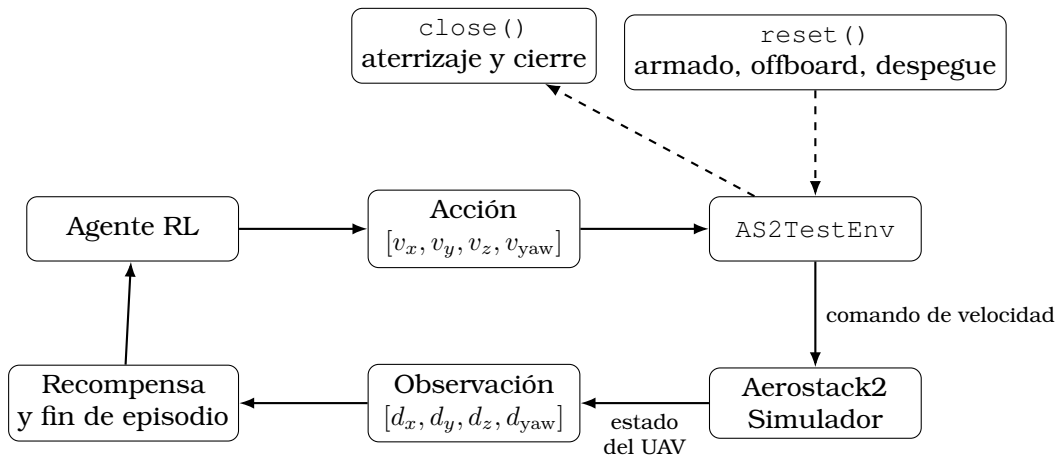


Figura 3.2: Ciclo de interacción entre el agente, la interfaz Gymnasium y el simulador durante un episodio.

Aerostack2 permanece encapsulada dentro de este componente de la arquitectura.

El método `reset` establece el estado inicial de un episodio. Su diseño incluye el armado del UAV, la activación del modo *offboard*, el despegue y la devolución de la primera observación junto con la información auxiliar del episodio. Por tanto, el reinicio no se limita a reinicializar variables internas, sino que prepara al sistema simulado para aceptar comandos de velocidad desde el agente. Esta preparación es relevante porque el contrato de Gymnasium presupone que, tras `reset`, la interfaz está lista para ejecutar `step` sin operaciones adicionales por parte del algoritmo.

El método `step` constituye el núcleo de la interacción. En cada llamada, el entorno recibe la acción continua del agente, la transforma en una orden de velocidad enviada mediante `SpeedMotion`, actualiza la observación relativa y calcula la recompensa y los indicadores de fin de episodio. El resultado sigue el contrato de Gymnasium y devuelve observación, recompensa, indicador de terminación, indicador de truncamiento e información auxiliar. La separación entre *terminación* y *truncamiento* permite distinguir un episodio que acaba por alcanzar una condición del problema de otro que finaliza por límite externo de pasos.

El método `close` se diseña como una operación segura de cierre del entorno. Su responsabilidad es aterrizar el UAV y liberar o cerrar los recursos asociados a la comunicación con Aerostack2. Esta decisión busca reducir el riesgo de que el final de un experimento deje el simulador en un estado inconsistente y concentra la lógica de apagado en el mismo componente que gestiona la interacción durante el episodio.

El entorno contempla también un modo de diseño denominado `randomize_hover_start`. Cuando está activo, el inicio, el objetivo y la guiñada del episodio se muestrean dentro de límites configurables y se evitan casos triviales mediante una separación mínima entre inicio y objetivo. Además se incorpora un margen de muestreo respecto a las fronteras de la escena y un límite máximo de distancia inicio-objetivo. Esta decisión evita que las observaciones relativas se saturen en $[-1, 1]$ desde el inicio del episodio y reduce la probabilidad de que un agente no entrenado genere inmediatamente una terminación fuera de límites.

La Figura 3.2 muestra el ciclo de interacción que se repite durante un episodio. A

partir de este ciclo, el Cuadro 3.1 sintetiza el contrato principal que la interfaz ofrece al algoritmo de aprendizaje.

Elemento	Diseño adoptado
Observación	Vector relativo normalizado $[d_x, d_y, d_z, d_{yaw}]$
Acción	Velocidades continuas $[v_x, v_y, v_z, v_{yaw}]$
Recompensa	Distancia normalizada al objetivo y bonificación terminal
Finalización	Éxito, salida de límites, baja altitud insegura o truncamiento temporal

Cuadro 3.1: Resumen del contrato principal de la interfaz Gymnasium.

3.3. Diseño del reinicio de episodios y accionabilidad

El reinicio de episodios constituye una decisión de diseño central en la arquitectura de aprendizaje por refuerzo para robótica aérea. A diferencia de una interfaz puramente numérico, no basta con asignar un nuevo estado inicial se debe ser coherentes el estado físico del multirrotor, el controlador, la plataforma y las lecturas que recibe la interfaz. Por este motivo, `reset()` se define como una precondition de la interacción posterior y no como una operación auxiliar.

El diseño combina una pose inicial fija, o `fixed_start_pose`, con dos vías de recuperación. La vía ordinaria devuelve al UAV que ya está en vuelo mediante comandos de velocidad acotados, evitando introducir operaciones bloqueantes entre episodios. Cuando esa maniobra no puede garantizar un estado válido, se recurre a un reinicio gestionado por el simulador. Esta segunda vía no se concibe como una recolocación ciega, sino como una recuperación capaz de restaurar de forma coherente el estado necesario para continuar el control.

El criterio de aceptación es independiente de la vía empleada, el UAV debe alcanzar y mantener la pose inicial, y el canal de control debe producir un movimiento físico medible. La aceptación de una orden por la interfaz robótica no basta para demostrar su efectividad. Si la interfaz generase observaciones y recompensas mientras las acciones no modifican el estado del vehículo, la experiencia dejaría de representar la tarea de aprendizaje. Por ello, los fallos de reinicio o de accionabilidad se tratan como condiciones de rechazo temprano, con diagnóstico explícito, y no como episodios válidos. La materialización de esta estrategia en el simulador y en `AS2TestEnv` se describe en la Sección 4.5.

3.4. Diseño del protocolo de evaluación de políticas

Estado o fallo	Significado de diseño	Respuesta prevista
reset_timeout	El UAV no alcanza la pose inicial dentro del tiempo definido.	Fallo acotado del reinicio y rechazo de continuar el entrenamiento.
post_controller_reacquire_timeout	Tras recrear o estabilizar el controlador, no se recupera la tolerancia de posición.	Diagnóstico específico del acoplamiento entre reset y controlador.
post_controller_hold_timeout	La pose se alcanza de forma transitoria, pero no se mantiene estable.	Rechazo del reset para evitar datos de episodio físicamente inválidos.
Comando aceptado sin movimiento	La interfaz acepta la orden, pero no se observa desplazamiento físico suficiente.	Fallo cerrado de accionabilidad antes de iniciar o continuar el entrenamiento.
Servicio de reset aplicado sin accionabilidad	El simulador informa que la pose se ha restaurado, pero el camino de comandos posterior no produce desplazamiento suficiente.	Reintento completo del servicio y, si persiste, parada explícita del entrenamiento.

Cuadro 3.2: Estados y fallos considerados en el diseño del reinicio de episodios.

El Cuadro 3.2 resume los principales estados de fallo que guían el diseño. En todos los casos se favorece un fallo explícito y trazable frente a un bloqueo indefinido o a la continuación de un entrenamiento con datos contaminados por un estado físico no válido.

3.4. Diseño del protocolo de evaluación de políticas

La evaluación de una política aprendida exige que los episodios se ejecuten bajo condiciones reproducibles. El protocolo se diseña en torno a dos decisiones. La primera es la reproducibilidad por semillas: el episodio i se genera reiniciando el entorno con la semilla $s_0 + i$, de modo que la secuencia de pares inicio-objetivo puede repetirse en evaluaciones posteriores. La segunda es fijar una interfaz de evaluación común para una política PPO en sus modos determinista y estocástico, que transforma la misma observación relativa normalizada en acciones dentro del espacio acotado del entorno. Cada evaluación registra métricas por episodio, como éxito, razón de terminación, pasos, distancia final, longitud de trayectoria, error de guiñada terminal y normas de acción, y las agrega globalmente y por tramos de distancia inicial. Así se caracteriza el comportamiento de la política bajo condiciones cuantitativas homogéneas.

Capítulo 4

Implementación de la arquitectura

Este capítulo describe cómo se materializa en el repositorio el diseño definido en el capítulo anterior. La atención se desplaza desde la formulación del problema hacia la organización del software, la integración efectiva con ROS 2 y Aerostack2, la configuración de PPO y los mecanismos de diagnóstico necesarios para ejecutar experimentos reproducibles.

La implementación construye una infraestructura de entrenamiento en la que Aerostack2 conserva el control de bajo nivel, el armado, el despegue y la ejecución simulada, mientras que la política aprendida actúa mediante comandos de velocidad compatibles con la plataforma. Esta separación evita atribuir a la política responsabilidades que, en la solución desarrollada, pertenecen al *stack* robótico.

4.1. Organización del software

El código desarrollado se distribuye en dos repositorios públicos. El repositorio principal `rl_uav_aerostack2`¹ contiene los componentes de la arquitectura de entrenamiento, incluida la interfaz Gymnasium, las configuraciones de entrenamiento, los scripts de ejecución y las utilidades de validación. El repositorio `as2_platform_multirotor_simulator` contiene el fork del simulador empleado para añadir el servicio de reinicio determinista. Esta separación hace explícito qué parte de la arquitectura corresponde a la interfaz Gymnasium y qué parte modifica la capa física de simulación.

La implementación se organiza alrededor de tres puntos de entrada. El módulo `rl_uav/envs/as2_test_env.py` contiene la clase `AS2TestEnv` y concentra la traducción entre Gymnasium y Aerostack2. El fichero `configs/train_ppo.yaml` define de forma declarativa los parámetros del entorno, la vectorización, las rutas de salida y los hiperparámetros de PPO. Por último, `scripts/train_ppo.py` lanza el entrenamiento o valida la configuración mediante un modo *dry-run*. Esta división permite cambiar una configuración experimental sin modificar la lógica de la interfaz Gymnasium ni los scripts de ejecución.

El repositorio incluye además utilidades de validación para comprobar la conexión monodron, la ejecución vectorizada, el inicio aleatorizado en hover y el comporta-

¹https://github.com/KafeisM/rl_uav_aerostack2

²https://github.com/KafeisM/as2_platform_multirotor_simulator

4.2. Integración con Aerostack2 y ROS 2

miento local con *mocks*. Estas comprobaciones no sustituyen a los experimentos, pero evitan confundir errores de infraestructura con errores de aprendizaje. En paralelo, el fork del simulador incorpora el servicio `ResetSimulatorState`, necesario para reiniciar de forma controlada el estado físico del multirrotor durante las campañas experimentales.

Elemento	Responsabilidad principal
<code>rl_uav/envs/as2_test_env.py</code>	Entorno Gymnasium y puente con Aerostack2.
<code>rl_uav/training/ppo_setup.py</code>	Construcción de entorno vectorizado, rutas de ejecución y modelo PPO.
<code>configs/train_ppo.yaml</code>	Configuración declarativa de entorno, entrenamiento e hiperparámetros.
<code>scripts/train_ppo.py</code>	Punto de entrada para entrenamiento y comprobación de configuración.
<code>scripts/as2_runtime.py</code>	Comprobación y relanzamiento controlado del simulador AS2.
<code>scripts/supervise_train.sh</code>	Supervisor de entrenamientos largos con reinicio del stack AS2 y reanudación desde <i>checkpoint</i> .
<code>rl_uav/evaluation/</code>	Protocolo de evaluación por semillas para políticas PPO en modos determinista y estocástico.
<code>scripts/evaluate_policy.py</code>	Punto de entrada de la evaluación formal de políticas entrenadas.
<code>as2_platform_multirrotor_simulator</code>	Fork del simulador con servicio de reinicio determinista del estado del multirrotor.
<code>as2_sim/launch_sim.bash</code>	Arranque del simulador monodron o multidron mediante nombres de dron.
<code>scripts/test_*.y</code> <code>scripts/validate_*</code>	Validaciones locales y reales de la interfaz Gymnasium, la vectorización y la integración.

Cuadro 4.1: Organización principal del software implementado.

El Cuadro 4.1 resume esta división. No se busca enumerar todos los ficheros del repositorio, sino mostrar que cada bloque tiene una responsabilidad acotada. La interfaz Gymnasium define la interacción, la configuración describe el experimento, los scripts ejecutan tareas reproducibles y las utilidades de validación reducen el riesgo de confundir un fallo de simulación con un fallo del algoritmo.

4.2. Integración con Aerostack2 y ROS 2

La integración implementada toma como punto de partida la infraestructura presentada en el Capítulo 2. ROS 2 aporta el grafo de comunicación robótica y Aerostack2 proporciona la capa aérea que encapsula operaciones como armado, modo *offboard*, despegue, aterrizaje y envío de velocidades [Fernández-Cortizas et al., 2023; Open Robotics, 2026]. En este capítulo no se repite su fundamento tecnológico; se describe cómo la interfaz `AS2TestEnv` utiliza esas interfaces para convertir acciones de PPO en órdenes ejecutables por el simulador.

La Figura 4.1 muestra la integración implementada. El agente PPO interactúa únicamente con la interfaz Gymnasium. La clase `AS2TestEnv` recibe las acciones, las convierte en comandos de velocidad y utiliza `DroneInterface` y `SpeedMotion` para comunicarse con Aerostack2. A su vez, Aerostack2 utiliza ROS 2 para transportar las órdenes y estados hacia el simulador multirrotor.

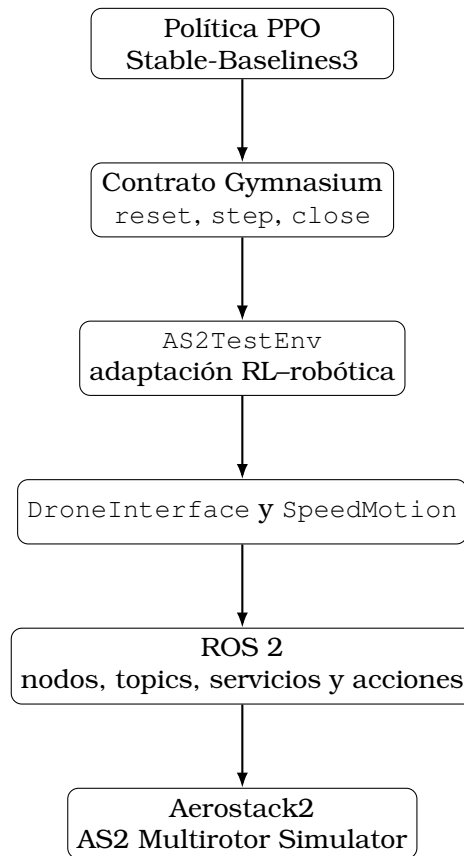


Figura 4.1: Integración implementada entre la política de aprendizaje, la interfaz Gymnasium, Aerostack2 y ROS 2.

En el código, la inicialización de ROS 2 se encapsula dentro de `AS2TestEnv`. El entorno protege la llamada a `rclpy.init()` con un bloqueo y una bandera de clase para evitar inicializaciones repetidas dentro del mismo proceso. Esta decisión es especialmente relevante en escenarios vectorizados, donde varias instancias lógicas del entorno pueden coexistir y compartir recursos de ROS 2. Tras inicializar ROS 2, el entorno crea una instancia de `DroneInterface` asociada al espacio de nombres del dron, por ejemplo `drone0`, y una instancia de `SpeedMotion` para el envío de velocidades.

El método `reset` ejecuta las operaciones necesarias para preparar un episodio. Inicializa la interfaz si aún no existe, arma el dron, activa el modo *offboard*, ejecuta el despegue y genera la primera observación. El método `step` aplica la acción emitida por el agente como un comando de velocidad lineal y velocidad de guiñada mediante `send_speed_command_with_yaw_speed`. Se emplea el marco de referencia *earth*, de forma coherente con la formulación del entorno como navegación local hacia un objetivo relativo. El método `close`, por su parte, realiza un cierre seguro mediante aterrizaje, paso a modo manual y liberación de la interfaz del dron. No se apaga globalmente `rclpy` desde cada instancia, porque hacerlo podría invalidar otras instancias activas del entorno.

El simulador se lanza desde `as2_sim/launch_sim.bash`. Este script permite seleccionar el número de drones y genera nombres de espacio como `drone0`, `drone1`, etc. Para la ejecución multidron se adopta un criterio de un único simulador compar-

4.3. Herramientas de visualización y monitorización

tido con varios espacios de nombres ROS 2, en lugar de iniciar varios simuladores independientes. Esta decisión reduce la sobrecarga de infraestructura y se alinea con la idea de Aerostack2 como marco capaz de operar sistemas aéreos multirrobot [Fernández-Cortizas et al., 2023]. Para hacer más robusta la ejecución, `scripts/as2_runtime.py` comprueba la existencia de las sesiones del simulador y permite relanzarlas antes de ejecutar validaciones que dependen de AS2.

4.3. Herramientas de visualización y monitorización

La implementación se apoya en herramientas de visualización y monitorización que permiten distinguir entre el funcionamiento del algoritmo, el estado del simulador y la respuesta física del UAV. Esta separación es necesaria porque un entrenamiento puede fallar por una configuración de PPO inadecuada, un reinicio inválido del episodio, un problema de comunicación ROS 2 o una orden aceptada por la interfaz que no produce movimiento físico.

RViz2 se utiliza como herramienta de inspección cualitativa del sistema robótico. Aunque el simulador de multirrotores de Aerostack2 no requiere interfaz gráfica, RViz2 permite observar la posición del UAV, la trayectoria reciente y los marcadores publicados para representar el objetivo. Esta visualización no se usa como métrica de aprendizaje, pero sí como apoyo para detectar comportamientos anómalos, como ausencia de movimiento, trayectorias divergentes o incoherencias entre el objetivo configurado y la escena simulada.

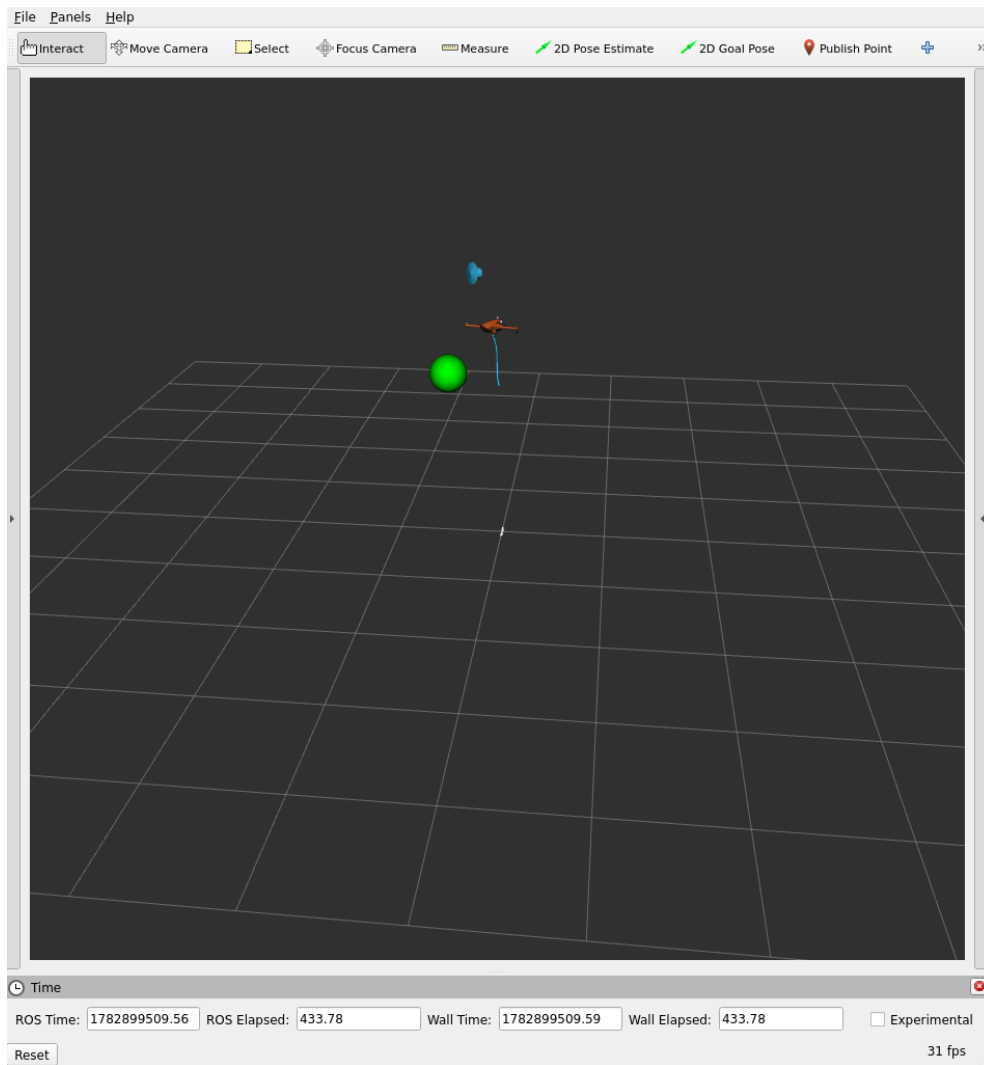


Figura 4.2: Visualización en RViz2 del UAV simulado, la trayectoria reciente y el marcador del objetivo durante una ejecución con Aerostack2.

La Figura 4.2 muestra un ejemplo de la visualización empleada durante las pruebas. El marcador verde representa el objetivo de navegación y el modelo del UAV permite comprobar de forma cualitativa si el movimiento observado es coherente con las acciones enviadas por la interfaz.

TensorBoard se emplea como herramienta de monitorización de las métricas emitidas por Stable-Baselines3 y por los monitores vectorizados. En las ejecuciones en las que PPO alcanza suficientes pasos para completar una primera actualización, esta herramienta permite inspeccionar magnitudes como recompensa media por episodio, longitud media de episodio y otros escalares registrados. Sin embargo, la interpretación de TensorBoard debe realizarse junto con los ficheros de monitor y los diagnósticos de la interfaz. Una curva ausente o congelada puede indicar que el entrenamiento no ha alcanzado un *rollout* completo, que el proceso se ha bloqueado durante un reset o que el simulador ha dejado de producir transiciones válidas.

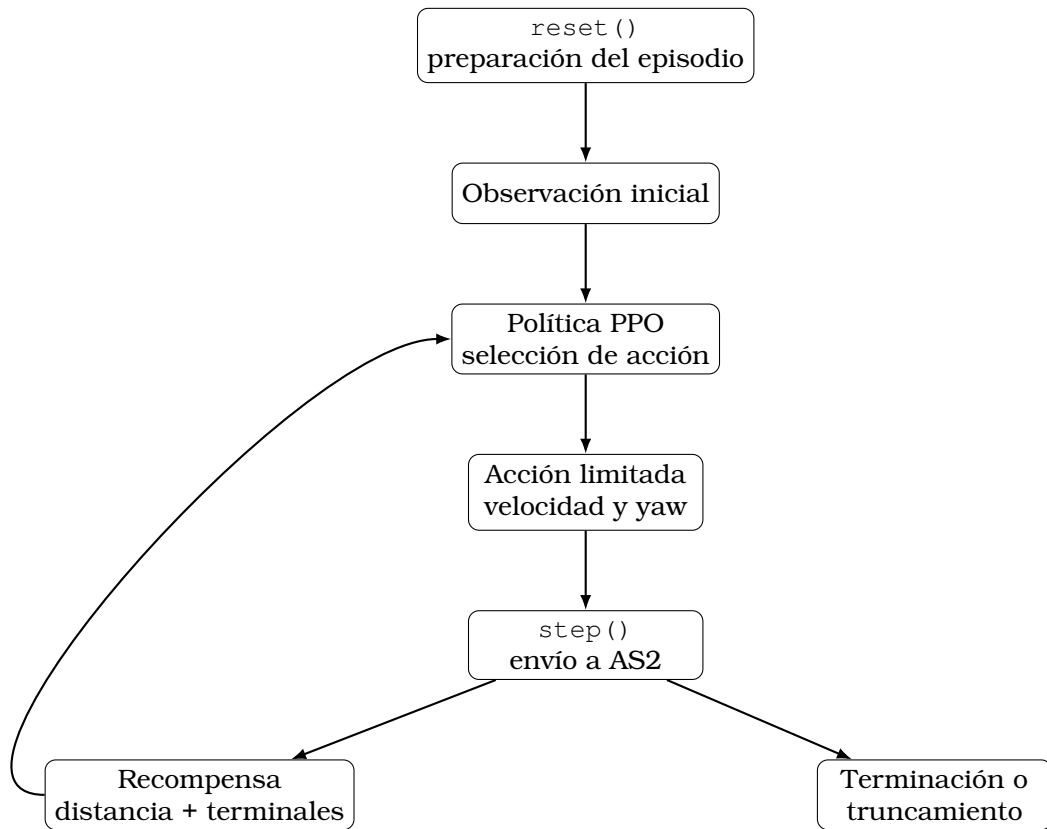


Figura 4.3: Ciclo implementado de un episodio de aprendizaje por refuerzo sobre AS2TestEnv.

4.4. Implementación de la interfaz Gymnasium

La clase `AS2TestEnv` materializa el diseño del Capítulo 3. En lugar de volver a justificar los espacios de observación y acción, esta sección se centra en cómo se ejecuta el contrato en el código. La interfaz registra `AS2TestEnv-v0`, define los espacios `Box` correspondientes, aplica los límites configurados y encapsula las operaciones de ciclo de vida que requiere `Aerostack2`.

Durante `reset`, la interfaz prepara la conexión robótica, arma el UAV, activa el modo *offboard*, ejecuta el despegue y devuelve la primera observación normalizada. Cuando se habilita `randomize_hover_start`, la posición inicial, la guiñada y el objetivo se muestrean dentro de los límites configurados. También se aplican márgenes en XY y altura, junto con `max_start_target_distance`, para evitar episodios que comiencen junto a fronteras problemáticas o con la observación saturada desde el primer paso.

Durante `step`, la acción recibida se recorta mediante `max_vel` y `max_yaw_vel`, se envía a `Aerostack2` con `SpeedMotion` y se lee el estado actualizado del UAV. Con esa información se calcula la recompensa, se evalúan las condiciones de terminación y se generan campos de diagnóstico.

La Figura 4.3 resume el flujo implementado. La responsabilidad principal de la interfaz no es definir de nuevo el problema de aprendizaje, sino convertir cada interacción de PPO en una operación robótica acotada, observable y registrable.

4.5. Reinicio determinista del simulador y validación de la accionabilidad

El mecanismo de reinicio constituye la parte más delicada de la integración. El reinicio habitual de Aerostack2 permite preparar el vehículo para el vuelo, pero no proporciona una operación atómica que restaure simultáneamente la dinámica del simulador, los estados auxiliares y la capa de plataforma tras el final de un episodio. En particular, restituir únicamente la pose puede conservar velocidades, referencias, estados del controlador o una máquina de estados de la plataforma que ya no reenvía las órdenes de movimiento. El resultado es un UAV aparentemente situado en el punto inicial, pero que no representa un estado inicial válido para la arquitectura de entrenamiento.

Para resolver esta limitación se extiende el simulador mediante el servicio ROS 2 `ResetSimulatorState`, expuesto bajo el espacio de nombres de cada vehículo, por ejemplo, `/drone0/platform/reset_simulator_state`. El servicio recibe la pose inicial y las tolerancias de posición, guiñada y velocidad que definen un reinicio aceptable. Su implementación valida la petición y sustituye el estado cinemático del multirrotor por la pose solicitada con velocidades, aceleraciones, fuerzas, pares y estados de los motores nulos. A continuación reinicializa el controlador, elimina las referencias pendientes de posición, velocidad, trayectoria y guiñada, y vuelve a establecer la base temporal de los bucles del simulador. Esta última operación evita que el primer paso posterior acumule el intervalo transcurrido durante el reinicio.

La extensión también sincroniza explícitamente la plataforma Aerostack2 con el estado físico restaurado. Tras reinicializar la plataforma, la deja conectada, armada, en modo *offboard* y en estado de vuelo, y actualiza el modo de control publicado para que la siguiente solicitud de control de velocidad no se descarte como si ya estuviera aplicada. Esta sincronización es necesaria porque la aceptación de una orden por ROS 2 no implica que la referencia alcance el simulador: la máquina de estados de la plataforma puede bloquear su reenvío aun cuando la dinámica ya se haya restaurado.

En `AS2TestEnv`, el cliente del servicio se crea con el mismo espacio de nombres que la interfaz. El retorno a la pose inicial se realiza normalmente con comandos de velocidad acotados, cuya autoridad se configura de forma independiente mediante `reset_max_vel`, así se mantiene un reinicio eficiente cuando el vehículo ya está en vuelo y el canal de control está sano. El servicio se emplea como vía de recuperación cuando el estado terminal impide esa maniobra o cuando el retorno por velocidad no converge dentro de los límites establecidos. Antes de intentar este retorno, un vehículo que permanece a baja altitud se eleva hasta una altura de seguridad configurada, sin ejecutar un ciclo interno de aterrizaje y despegue que pudiera dejar la plataforma en un estado incompatible con el control *offboard*.

La respuesta del servicio no cierra por sí sola el reinicio. La interfaz comprueba los errores residuales y las marcas de restauración de la dinámica, del controlador, de la odometría y de la plataforma; después confirma la pose y verifica el canal de movimiento mediante una sonda de accionabilidad. Los reintentos se acotan mediante `reset_service_max_attempts` y `reset_service_retry_backoff_s`. Si alguna de estas comprobaciones falla, `reset()` termina con un diagnóstico explícito en lugar de iniciar un episodio cuyas acciones no puedan modificar el estado del UAV. De es-

te modo, la capa añadida transforma el reinicio del simulador en una precondition verificable del contrato Gymnasium, no en una mera recolocación visual del vehículo.

4.6. Infraestructura de entrenamiento y vectorización

La infraestructura de entrenamiento se concentra en `ppo_setup.py`. Este módulo carga la configuración, crea los directorios de salida, instancia los entornos, envuelve la ejecución con `VecMonitor` y construye el modelo PPO. El registro de métricas se realiza sobre el entorno vectorizado para obtener estadísticas de episodio agregadas sin duplicar lógica de monitorización en cada instancia.

La vectorización se implementa mediante espacios de nombres ROS 2. A partir del prefijo `drone` se generan instancias como `drone0`, `drone1`, `drone2`, ..., `droneN`, todas conectadas al mismo simulador compartido. Esta solución evita lanzar un simulador por entorno y mantiene la correspondencia entre instancia de Gymnasium y UAV simulado. La implementación puede usar `DummyVecEnv` o `SubprocVecEnv` según la configuración, aunque la validación inicial prioriza ejecuciones controladas antes que paralelización agresiva.

El entrenamiento se ejecuta en dos pasos explícitos. Primero se lanza o verifica el simulador con los scripts de AS2; después se ejecuta `scripts/train_ppo.py` con la configuración seleccionada. Para campañas largas, `train_ppo.py` acepta `-resume-from`, que reanuda desde un *checkpoint* explícito o desde el más reciente y calcula el número de pasos restante hasta el total configurado (Stable-Baselines3 suma el argumento al contador del *checkpoint*, por lo que pasar el total completo duplicaría el objetivo en cada reanudación).

El supervisor `scripts/supervise_train.sh` automatiza tres mecanismos sobre esa base. Ante un fallo del entrenamiento, reinicia el stack AS2 completo, espera a que el servicio de reinicio de cada dron esté disponible y reanuda desde el último *checkpoint*. Mediante la variable `RECYCLE_SECONDS` aplica además un reciclado preventivo donde interrumpe el entrenamiento a intervalos fijos y lo reanuda sobre un stack recién iniciado, de modo que cada tramo se ejecuta sobre un simulador sin degradación acumulada; los reciclados se detectan por el tiempo transcurrido, no solo por el código de salida, y no consumen el presupuesto de reintentos por fallo. Por último, `DRONE_NAMESPACES` extiende el reinicio y la comprobación de salud a varios drones para las campañas vectorizadas, e `INITIAL_RESUME` permite sembrar una etapa del curriculum con el modelo final de la etapa anterior cuando aún no existen *checkpoints* propios.

La validación inicial confirma que el pipeline puede crear el modelo PPO, conectarse con AS2, enviar comandos de velocidad y generar trazas monitorizables. Esta evidencia prueba la infraestructura de entrenamiento, no el rendimiento final de la política; esa interpretación corresponde a los capítulos de pruebas y resultados.

4.7. Verificación técnica de la implementación

Las comprobaciones de esta sección verifican que los componentes desarrollados materializan correctamente los contratos definidos en los capítulos anteriores. Su objeto es comprobar la calidad de la implementación, que la interfaz Gymnasium conserva

Implementación de la arquitectura

su semántica, que la configuración se propaga sin ambigüedades, que la integración robótica mantiene un ciclo de episodio válido y que la infraestructura deja artefactos diagnósticos. No se pretende determinar si una política aprende a resolver la tarea de navegación; esa validación aplicada se presenta en el Capítulo 5.

La verificación se estructura en tres niveles complementarios. Las pruebas de encapsulamiento aíslan la lógica de la interfaz Gymnasium y de la configuración mediante dobles de prueba. Las comprobaciones de integración ejercitan las interfaces con ROS 2, Aerostack2 y el simulador. Por último, las ejecuciones de humo verifican que el proceso de entrenamiento puede instanciar sus componentes y producir los artefactos previstos, sin interpretar sus métricas como rendimiento de la política. El Cuadro 4.2 resume la finalidad de cada nivel.

Nivel	Contrato verificado	Evidencia esperada
Pruebas de encapsulamiento	Semántica de la API Gymnasium, configuración y mecanismos de protección.	Propiedades reproducibles comprobadas sin depender del simulador.
Comprobaciones de integración	Ciclo de vida del UAV, reinicio y transmisión de comandos de velocidad.	Estado físico consistente, movimiento medible y diagnósticos acotados.
Ejecuciones de humo	Cableado del proceso de entrenamiento, monitorización y persistencia.	Directorios de salida, ficheros de monitor, <i>checkpoints</i> y errores clasificables.

Cuadro 4.2: Niveles de verificación técnica aplicados a la implementación.

4.7.1. Pruebas de encapsulamiento y configuración

Las pruebas de encapsulamiento cubren las propiedades que deben mantenerse con independencia del estado de los procesos externos. Se ejecutan con dobles de prueba y validan la API Gymnasium, la configuración declarativa y los mecanismos de seguridad que modifican las acciones. De este modo, un cambio en la lógica de la interfaz Gymnasium puede aislarse antes de atribuir una incidencia a ROS 2, a Aerostack2 o al algoritmo de aprendizaje.

4.7. Verificación técnica de la implementación

Validador	Propósito	Criterio de aceptación
<code>scripts/test_vectorization.py</code>	Comprobar el contrato de la interfaz Gymnasium, la vectorización, el reinicio y los diagnósticos mediante dobles de prueba.	La API, los espacios y los campos de diagnóstico conservan la semántica esperada.
<code>scripts/test_ppo_config_propagation.py</code>	Verificar que los parámetros YAML llegan a la interfaz Gymnasium y al modelo PPO.	Los parámetros críticos aparecen en las instancias finales sin valores implícitos no trazables.
<code>scripts/test_resume_checkpoint.py</code>	Validar la resolución de <i>checkpoints</i> para reanudación de PPO.	Se resuelve una ruta explícita o el último <i>checkpoint</i> disponible de forma predecible.
<code>scripts/test_bounds_action_guard.py</code> y <code>scripts/test_predictive_guard.py</code>	Verificar los guards de acción reactivos y predictivos en fronteras y techo.	Los comandos salientes se sustituyen por empuje interior en las condiciones geométricas especificadas.
<code>scripts/test_idle_watchdog.py</code>	Validar el mecanismo de vigilancia de comandos inactivos.	Publica hover tras el umbral de inactividad, no interfiere con la operación normal y se desactiva por defecto.

Cuadro 4.3: Pruebas de encapsulamiento representativas de la implementación.

El Cuadro 4.3 recoge los comprobadores que preservan los invariantes de implementación más relevantes. Los contratos asociados a una campaña concreta de entrenamiento se mantienen fuera de esta sección, pues su objeto es validar una aplicación de la infraestructura y no sus componentes generales.

4.7.2. Comprobaciones de integración con Aerostack2

Las comprobaciones de integración verifican que el contrato de Gymnasium se mantiene al atravesar la frontera entre la interfaz Python y el sistema robótico simulado. Se ejercitan los métodos `reset`, `step` y `close` sobre Aerostack2 para comprobar, respectivamente, la preparación del estado inicial, la transmisión de referencias de velocidad y la liberación segura de recursos. El objetivo no es medir la calidad de una trayectoria aprendida, sino asegurar que las transiciones que recibiría una política son físicamente coherentes y observables.

El reinicio se verifica en dos fronteras. La primera corresponde a la respuesta del servicio del simulador, descrito en la Sección 4.5, que acredita la restauración inmediata del estado. La segunda corresponde a la telemetría posterior de ROS 2 y al canal de movimiento, que confirma que la plataforma permanece preparada para aplicar la siguiente referencia. Esta separación evita aceptar como válido un reinicio que solo ha modificado la pose visible del UAV.

El Cuadro 4.4 resume los contratos de integración que se comprueban y la respuesta de la implementación cuando alguno de ellos no se satisface.

Implementación de la arquitectura

Contrato de integración	Comprobación	Respuesta ante incumplimiento
Estado inicial	La pose, el modo de operación y la estabilidad se contrastan tras <code>reset()</code> .	Se rechaza el episodio y se emite un diagnóstico acotado.
Estado del simulador	La respuesta del servicio y la telemetría posterior confirman la restauración y la ausencia de artefactos transitorios.	Se reintenta la recuperación dentro de los límites configurados o se detiene el ciclo.
Accionabilidad	Una referencia enviada mediante <code>step()</code> debe producir desplazamiento físico medible.	Se clasifica el canal como no operativo y no se generan transiciones válidas.
Cierre del episodio	<code>close()</code> debe cancelar referencias activas y liberar los recursos creados por la interfaz.	Se informa el fallo de cierre para impedir que afecte a la siguiente ejecución.

Cuadro 4.4: Contratos verificados en la integración con Aerostack2.

El repositorio incorpora además comprobaciones específicas para el servicio de reinicio. Estas verifican la disponibilidad de la interfaz, el rechazo de peticiones no válidas y la coherencia del estado que queda preparado para el siguiente comando de velocidad. En conjunto, estas pruebas establecen que la integración está en condiciones de proporcionar episodios válidos; la aplicación de dichos episodios a una tarea de control se reserva para el Capítulo 5.

4.8. Alcance de la verificación de implementación

Las comprobaciones anteriores establecen tres propiedades de la implementación. En primer lugar, la interfaz Gymnasium conserva su contrato y recibe una configuración trazable. En segundo lugar, la integración con Aerostack2 permite inicializar el vehículo, aplicar comandos de velocidad, recuperar un estado inicial verificable y cerrar el ciclo de episodio sin dejar referencias activas. Por último, la infraestructura de entrenamiento puede crear sus directorios de salida, registrar diagnósticos y conservar *checkpoints*. Estas propiedades demuestran que los componentes de la arquitectura están conectados y se comportan de forma controlada.

Este alcance no equivale a demostrar que PPO resuelva la tarea de navegación ni a valorar la calidad de las trayectorias generadas. La verificación de implementación responde a la pregunta de si la arquitectura puede producir transiciones físicas válidas y trazables. El Capítulo 5 utiliza esa infraestructura en un entrenamiento de control de velocidad para evaluar el comportamiento de la política sobre un problema concreto. Así, la evidencia de correctitud de implementación queda separada de la evidencia de utilidad del control aprendido.

Capítulo 5

Aplicación y validación de la arquitectura

Este capítulo aplica la infraestructura desarrollada a una tarea concreta de control de velocidad para un UAV simulado. Mientras que el Capítulo 4 verifica que los componentes están correctamente implementados y pueden producir transiciones físicas válidas, aquí se analiza si la arquitectura permite entrenar una política PPO para alcanzar un objetivo relativo mediante comandos continuos de velocidad y guiñada. Esta aplicación convierte la arquitectura en una infraestructura de experimentación: las observaciones, las acciones, la recompensa, el reinicio y la monitorización se ejercitan de forma conjunta bajo una misma tarea de control.

La validación distingue la disponibilidad de la infraestructura del comportamiento de la política. Un entrenamiento solo es interpretable si cada episodio comienza en un estado verificable y las acciones modifican físicamente el estado del vehículo; sobre esa condición se pueden evaluar la aproximación al objetivo, las terminaciones y las métricas de aprendizaje. Las secciones siguientes presentan la formulación de la tarea, las salvaguardas y la configuración empleada, así como los criterios con los que se analizan las ejecuciones.

5.1. Propósito y alcance de la aplicación

La aplicación de validación consiste en entrenar una política PPO que recibe el estado relativo del UAV respecto de un objetivo y emite velocidades lineales y de guiñada. El entrenamiento se ejecuta sobre `AS2TestEnv`, que traduce cada acción de la política en una referencia para `Aerostack2` y devuelve la observación, la recompensa y el estado de terminación correspondientes. De este modo, la tarea exige simultáneamente que el canal de control sea accionable, que el ciclo de reinicio sea fiable y que la formulación de aprendizaje proporcione una señal útil para la navegación local.

Esta aplicación no se plantea como una demostración aislada de que el proceso de entrenamiento se inicia, sino como una validación integrada del objetivo del trabajo. Una ejecución adecuada debe generar episodios trazables, conservar las condiciones de seguridad y permitir evaluar si la política reduce la distancia al objetivo y alcanza la región de éxito. Los apartados posteriores concretan esta tarea mediante la formulación de control, la recompensa, el curriculum, las protecciones de frontera, la

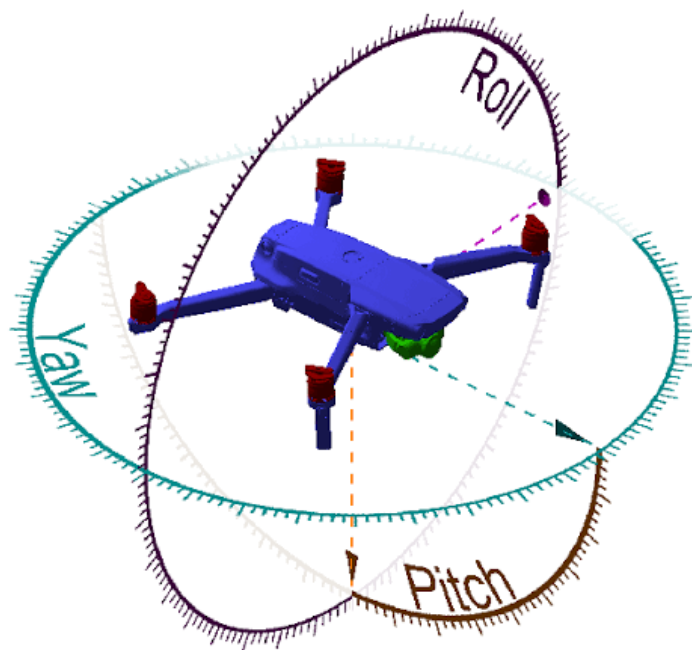


Figura 5.1: Representación conceptual de la pose de un dron mediante los ángulos de *roll*, *pitch* y *yaw*.
Fuente: ResearchGate.

configuración de PPO y las métricas de aceptación.

5.2. Formulación de la tarea de control

El problema de control se formula como una tarea de navegación local hacia un objetivo relativo. En este contexto, navegación local significa que el agente no recibe una misión global completa ni una trayectoria predefinida con todos sus puntos intermedios, sino la relación geométrica entre el estado actual del UAV y un objetivo inmediato. A partir de esa relación, el agente debe emitir comandos de velocidad que acerquen el vehículo al objetivo y mantengan una orientación coherente con la dirección de desplazamiento.

Antes de definir formalmente las observaciones y acciones, conviene distinguir las magnitudes básicas que describen el movimiento de un multirrotor. La posición indica dónde se encuentra el UAV en el espacio; la velocidad indica cómo cambia esa posición; y la orientación describe la rotación del vehículo respecto a sus ejes. En un dron se suelen emplear tres ángulos principales, denominados *roll*, *pitch* y *yaw*. El *roll* representa la inclinación lateral, el *pitch* la inclinación hacia delante o hacia atrás, y el *yaw* la rotación alrededor del eje vertical. La Figura 5.1 resume esta idea de pose del dron mediante una representación conceptual tomada de ResearchGate.¹

En el presente trabajo no se controla directamente la actitud completa del dron ni las velocidades de giro de cada rotor. La acción se define a un nivel de abstracción superior, en el que el agente genera velocidades lineales y una velocidad de guiñada.

¹Imagen disponible en ResearchGate: https://www.researchgate.net/figure/A-conceptual-representation-of-drone-pose-A-drone-here-represented-by-a-DJI-Mavic-2-is_fig1_356985671. Según la página consultada, la figura aparece con licencia Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International.

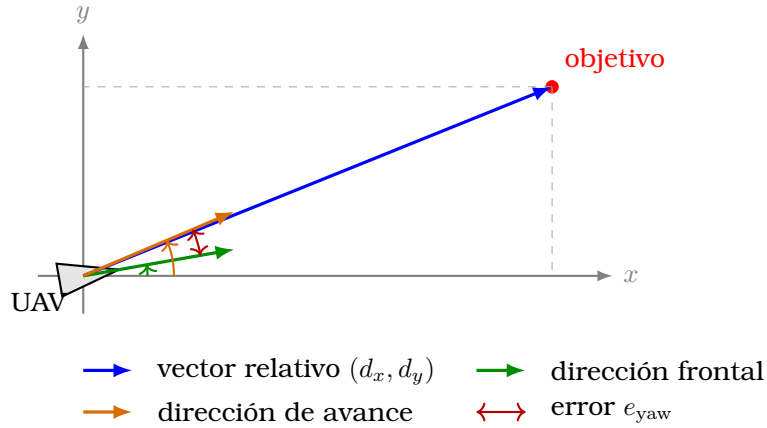


Figura 5.2: Relación geométrica entre el UAV, el objetivo relativo, la dirección de avance y la orientación de guiñada utilizada en el criterio de *path-facing*.

Esta elección sitúa el aprendizaje por refuerzo por encima del control de bajo nivel y delega en Aerostack2 la ejecución de los comandos sobre el simulador. Desde el punto de vista del diseño, esta separación reduce la complejidad del problema aprendido y mantiene la compatibilidad con la interfaz de control de velocidad disponible.

La formulación distingue también entre la dirección frontal del UAV y la dirección real de avance. Un dron puede desplazarse hacia un objetivo aunque su eje frontal no esté alineado con la trayectoria seguida. El criterio de *path-facing*, ilustra la diferencia entre ambas magnitudes y la relación con el error de guiñada. En la formulación final de la solución no se exige el seguimiento de una trayectoria: la guiñada se considera únicamente al evaluar el éxito terminal. La Figura 5.2 representa esta relación geométrica.

La acción se define mediante el vector continuo

$$a = [v_x, v_y, v_z, v_{yaw}].$$

Las tres primeras componentes representan velocidades lineales en los ejes cartesianos y la cuarta representa la velocidad angular de guiñada. El espacio de acciones se acota mediante los parámetros `max_vel` y `max_yaw_vel`, lo que introduce una restricción explícita de diseño sobre la magnitud de las órdenes que el agente puede emitir. Esta acotación es necesaria porque una política de aprendizaje por refuerzo puede proponer acciones arbitrarias dentro de su espacio de salida, mientras que el sistema físico o simulado solo debe recibir comandos dentro de un rango admisible.

La observación se define mediante el vector relativo

$$o = [d_x, d_y, d_z, d_{yaw}].$$

Cada componente se normaliza en el intervalo $[-1, 1]$. Las tres primeras componentes codifican el desplazamiento relativo hacia el objetivo, y la cuarta codifica la diferencia relativa de guiñada. La normalización reduce la dependencia respecto a la escala concreta del escenario y proporciona al agente entradas con rangos homogéneos, una condición práctica para estabilizar el tratamiento de las observaciones por parte del algoritmo de aprendizaje.

5.3. Recompensa y condiciones de finalización

La recompensa final usada en la fase de experimentación principal se diseña para mantener una señal mínima e interpretable. En cada paso se penaliza únicamente la distancia normalizada al objetivo. Para ello, si d_t representa la distancia euclídea entre el UAV y el objetivo, y $d_{\text{máx}}$ representa la distancia máxima de normalización. A partir de ambas magnitudes se obtiene

$$d_{\text{norm}} = \min\left(\frac{d_t}{d_{\text{máx}}}, 1\right), \quad r_{\text{dist}} = -d_{\text{norm}}. \quad (5.1)$$

En la interfaz Gymnasium implementada, $d_{\text{máx}}$ se calcula a partir del límite espacial de posición del escenario, de forma que la distancia normalizada queda acotada en el intervalo $[0, 1]$.

La interfaz Gymnasium conserva la capacidad de calcular términos adicionales, como progreso, seguridad vertical o coherencia de guiñada con la dirección de movimiento. Sin embargo, esos términos se desactivan a partir de Exp010 y en toda la campaña principal para evitar que el agente optimice una recompensa de shaping distinta de la tarea principal. La guiñada se mantiene como requisito terminal suave. Cuando el UAV alcanza la región objetivo, la bonificación de éxito se reduce en función del error de guiñada. Si e_{yaw} representa el error angular normalizado, la recompensa terminal de éxito se calcula mediante

$$r_{\text{success}} = R_{\text{success}} - w_{\text{yaw}} \frac{|e_{\text{yaw}}|}{\pi}. \quad (5.2)$$

La expresión final aplicada desde Exp010 en adelante incorpora las condiciones terminales del episodio de la forma siguiente.

$$r_t = \begin{cases} -P_{\text{oob}}, & \text{si el UAV sale de los límites del escenario,} \\ -P_{\text{oob}}, & \text{si el UAV entra en baja altitud insegura,} \\ r_{\text{success}}, & \text{si } d_t < d_{\text{threshold}}, \\ -d_{\text{norm}}, & \text{en el resto de pasos.} \end{cases} \quad (5.3)$$

El truncamiento por alcanzar `max_steps` no introduce una recompensa adicional, sino que limita la duración del episodio cuando no se ha alcanzado una condición terminal. El Cuadro 5.1 resume los componentes de esta formulación y su efecto esperado.

5.4. Curriculum de distancia

Con la recompensa mínima $-d_{\text{norm}}$, la única señal positiva del problema es la bonificación terminal, y su densidad depende de la probabilidad de que la exploración alcance la esfera de éxito de 0,4 m. Con separaciones inicio-objetivo de hasta 5 m esa probabilidad es demasiado baja para arrancar el gradiente con la tasa de aprendizaje configurada. El diseño adopta por ello un curriculum de distancia [Bengio et al., 2009] en tres etapas: la primera acota la separación máxima a 2,5 m, de modo que un vuelo directo requiere del orden de 25 pasos; las dos siguientes amplían el límite a 3,5 m y finalmente a 5,0 m. Cada etapa se inicializa con el modelo final de la anterior (*warm start*) y todas mantienen la separación mínima de 1,0 m, de forma que la distribución de episodios de una etapa contiene a la anterior y la política no olvida los objetivos cercanos al aprender los lejanos.

Componente	Expresión	Condición	Efecto esperado
Penalización por distancia	$r_{\text{dist}} = -d_{\text{norm}}$	En cada paso	Favorece que el UAV reduzca progresivamente la distancia al objetivo.
Bonificación por éxito	$R_{\text{success}} - w_{\text{yaw}} e_{\text{yaw}} /\pi$	Si $d_t < d_{\text{threshold}}$	Premia alcanzar la región objetivo y reduce la bonificación si la guiñada terminal es inadecuada.
Penalización por salida de límites	$-P_{\text{oob}}$	Si el UAV supera los límites del escenario	Desincentiva trayectorias inseguras o alejadas de la zona de entrenamiento.
Penalización por baja altitud insegura	$-P_{\text{oob}}$	Si el UAV desciende por debajo del umbral inseguro	Evita que el aprendizaje explote estados cercanos al suelo o físicamente no válidos.
Truncado temporal	Sin recompensa adicional	Si se alcanza el número máximo de pasos	Limita la duración del episodio cuando no se alcanza una condición terminal.

Cuadro 5.1: Componentes de la función de recompensa y condiciones de finalización de la interfaz Gymnasium.

5.5. Protecciones de frontera

La observación relativa tiene una consecuencia de diseño que los primeros experimentos hicieron explícita y es que el agente no puede percibir los límites del escenario. Una terminación por salida de límites es, desde el punto de vista de la política, un castigo imprevisible aplicado en estados que no se distinguen de los estados seguros, por lo que la función de valor no puede anticiparla. Además, cada una de estas terminaciones obliga a un reinicio mediante el servicio del simulador, con un coste de decenas de segundos y efectos acumulativos sobre la plataforma. Por ambos motivos, el diseño sustituye la muerte en frontera por una protección activa: un *guard* de acción que, cerca de los límites, reemplaza la componente de velocidad saliente por un empuje hacia el interior.

El *guard* se diseña con disparo predictivo. La versión reactiva, que actúa cuando la posición cruza una línea de guardia, resultó insuficiente en el eje vertical ya que la plataforma invierte la velocidad con lentitud y la inercia atraviesa el margen antes de que la orden correctora surta efecto. Por ello el disparo evalúa la posición *predicha*, calculada como posición actual más velocidad medida por un horizonte de anticipación, y el empuje vertical utiliza la autoridad completa de velocidad disponible. Las terminaciones por salida de límites y por baja altitud se conservan intactas como red de seguridad final, pero con el *guard* activo pasan de ser el desenlace dominante (80–93 % de los episodios en Exp010–Exp011) a ser residuales, lo que elimina a la vez el ruido inaprendible de la recompensa y la mayor parte de los reinicios costosos.

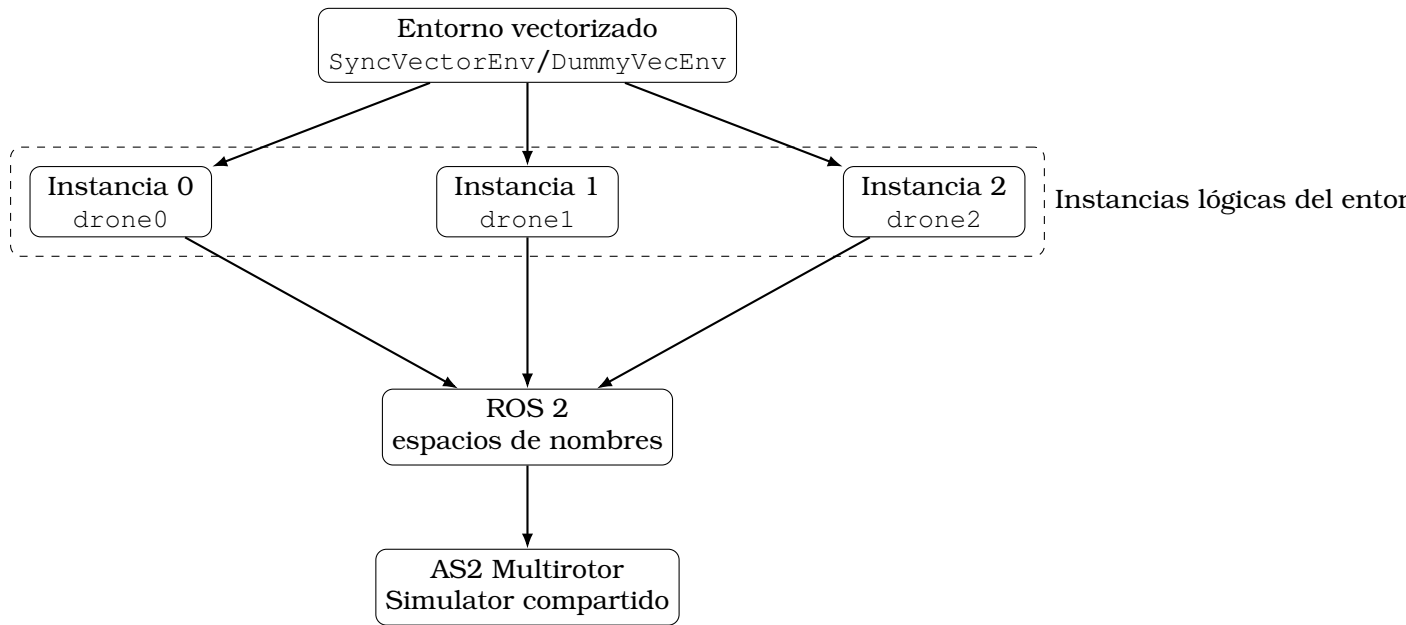


Figura 5.3: Extensión vectorizada mediante instancias lógicas asociadas a espacios de nombres diferenciados sobre una infraestructura de simulación compartida.

5.6. Extensión a ejecución vectorizada

La ejecución vectorizada se diseña para permitir varias instancias lógicas del entorno sobre un único simulador compartido. El diseño utiliza espacios de nombres diferenciados, `drone0` a `droneN`, de forma que cada UAV simulado pueda ser identificado y gobernado de manera independiente dentro de la misma infraestructura ROS 2. Esta organización reduce la necesidad de duplicar toda la infraestructura del simulador, pero exige tratar con cuidado la inicialización de ROS 2, los nombres de los drones y la sincronización entre episodios.

La Figura 5.3 resume esta organización. Cada instancia lógica mantiene su propio espacio de nombres, pero todas comparten la infraestructura física de simulación, por lo que sus reinicios y sus tiempos de paso deben coordinarse.

La elección del mecanismo de vectorización viene determinada por una propiedad del simulador: opera en tiempo de reloj real, de modo que cada paso del entorno consume la duración física configurada (0,2 s) con independencia del proceso que lo invoque. Esta propiedad invalida en este sistema el razonamiento habitual a favor de la vectorización secuencial (`SyncVectorEnv` en `Gymnasium`, `DummyVecEnv` en `Stable-Baselines3`), cuyo beneficio procede de agrupar las observaciones para una única inferencia por lotes: con un simulador de reloj real, N entornos secuenciales tardan N veces el paso físico en producir N muestras, es decir, el mismo caudal de muestreo que un solo entorno. Por ello el diseño adopta `SubprocVecEnv`, que ejecuta los pasos de los N entornos en paralelo real manteniendo la inferencia por lotes, con la expectativa (verificada experimentalmente) de multiplicar el muestreo por el número de drones.

El paso vectorizado en paralelo introduce, sin embargo, un modo de fallo propio de los simuladores de reloj real, el avance en *lockstep*. Cuando uno de los entornos

ejecuta su reinicio, es una operación que tarda decenas de segundos, el paso vectorizado queda bloqueado para todos; durante ese intervalo los demás drones no reciben órdenes nuevas mientras el mundo sigue avanzando, y el controlador de Aerostack2 mantiene la última referencia de velocidad como consigna activa. El diseño incorpora por ello un mecanismo de vigilancia (*watchdog*) de comandos inactivos, es decir, si un entorno en vuelo supera un umbral de tiempo sin emitir referencias de movimiento, el mecanismo publica una consigna de velocidad nula de forma periódica hasta que el flujo normal de comandos se reanuda. El umbral se dimensiona por encima de la duración del paso, de modo que el mecanismo permanece inerte durante la operación normal y solo actúa en los huecos de *lockstep*; en ejecución monodron queda desactivado y el comportamiento del entorno es idéntico al de la fase anterior.

5.7. Configuración del entrenamiento PPO

La configuración de PPO se materializa en `configs/train_ppo.yaml`. El Capítulo 2 justifica la elección del algoritmo; esta sección se limita a documentar cómo se parametriza su uso dentro de la arquitectura implementada y cómo se preserva la trazabilidad de cada ejecución.

La configuración se divide en tres bloques principales. El bloque `experiment` define el nombre del experimento, la semilla y el directorio raíz de salida. El bloque `environment` agrupa los parámetros del entorno, incluyendo el identificador `AS2TestEnv-v0`, el número de instancias vectorizadas, el prefijo de espacio de nombres, los límites de posición y velocidad, la duración de cada paso, el objetivo, las condiciones de éxito y las penalizaciones de seguridad. El bloque `ppo` contiene los hiperparámetros del algoritmo, como la tasa de aprendizaje, el número de pasos por actualización, el tamaño de lote, el factor de descuento, el rango de *clipping* y la arquitectura de las redes de política y valor.

Parámetro	Valor base	Función
<code>total_timesteps</code>	1000000	Duración máxima de la campaña de entrenamiento.
<code>learning_rate</code>	3.0e-05	Tamaño de actualización de los parámetros de PPO.
<code>n_steps</code>	512	Número de pasos recogidos antes de cada actualización.
<code>batch_size</code>	32	Tamaño de lote usado durante la optimización.
<code>clip_range</code>	0.2	Límite de variación entre política antigua y política nueva.
<code>use_sde</code>	<code>true</code>	Activación de exploración dependiente del estado.
<code>net_arch</code>	[128, 128]	Capas ocultas de las redes de política y valor.

Cuadro 5.2: Principales parámetros PPO definidos en la configuración base de entrenamiento.

El Cuadro 5.2 recoge los parámetros más representativos de la configuración base. Estos valores no deben interpretarse como una optimización definitiva del algoritmo, sino como una configuración inicial trazable y reproducible para validar la infraes-

estructura de entrenamiento. Los capítulos de pruebas y resultados son los que deben determinar si una configuración concreta produce aprendizaje útil, estabilidad suficiente o necesidad de ajuste.

A partir de Exp010 se utiliza una configuración ajustada con tasa de aprendizaje $3.0e-05$, `n_steps=512`, `batch_size=32`, `n_epochs=5`, `ent_coef=0.0`, exploración SDE y redes MLP de dos capas de 128 unidades por rama. Las configuraciones Exp010, Exp011 y Exp012 mantienen estos hiperparámetros y modifican principalmente propiedades del entorno y de la tarea, como la recompensa mínima, la randomización completa, los márgenes de muestreo, el límite de distancia inicio-objetivo, los guards de acción y la reanudación de entrenamiento. Esta separación evita mezclar ajustes del algoritmo con cambios en las condiciones de interacción del UAV.

La configuración también explicita decisiones de que afectan a la reproducibilidad. El entrenamiento se fija inicialmente en CPU, coherente con el uso de políticas MLP y con la recomendación habitual de Stable-Baselines3 para evitar sobrecostes de GPU en redes no convolucionales. Además, se separan los directorios de *logs*, monitorización y *checkpoints*, lo que permite analizar una ejecución sin depender únicamente de la salida estándar del proceso. Esta estructura facilita repetir experimentos, comparar configuraciones y aislar fallos entre simulación, interfaz Gymnasium y algoritmo.

5.8. Métricas y criterios de interpretación

Las métricas empleadas combinan indicadores de aprendizaje e indicadores de infraestructura. En el primer grupo se encuentran la recompensa por episodio, la longitud de episodio, la tasa de éxito y la distancia final al objetivo. En el segundo grupo se registran razones de terminación, desplazamiento físico, longitud de trayectoria, altitud mínima, ratio de comandos aceptados, método de reset y clase de fallo. Esta separación permite determinar si una ejecución fracasa porque la política todavía no resuelve la tarea o porque la arquitectura no está entregando transiciones físicamente válidas.

Las curvas de TensorBoard se utilizan como evidencia complementaria, no como única prueba de aprendizaje. En particular, las métricas `rollout/ep_rew_mean` y `rollout/ep_len_mean` permiten observar tendencias agregadas, mientras que los campos de monitor por episodio permiten explicar esas tendencias mediante razones de terminación, distancia final, éxitos, salidas de límites o eventos de baja altitud. Esta combinación es necesaria porque una curva plana de recompensa puede deberse a una política que no aprende, pero también a episodios dominados por terminaciones de seguridad o por reinicios costosos del simulador.

El criterio de aceptación para un entrenamiento no se reduce a que el proceso de Python siga ejecutándose. Una ejecución solo se considera apta para análisis de aprendizaje si la arquitectura establece episodios desde estados válidos, si las acciones producen movimiento físico medible, si los fallos de reset no se ocultan como episodios válidos y si las métricas de monitorización permiten reconstruir lo ocurrido. En caso contrario, el resultado se documenta como evidencia de integración y se utiliza para mejorar la arquitectura antes de extraer conclusiones sobre PPO.

Capítulo 6

Resultados de la aplicación y validación de la arquitectura

Este capítulo analiza los resultados de aplicar la arquitectura desarrollada a la tarea de control de velocidad descrita en el Capítulo 5. El objetivo es comprobar que la infraestructura permite ejecutar entrenamientos trazables y obtener evidencia sobre el comportamiento de una política PPO en condiciones controladas. Por ello, los resultados se organizan según las propiedades de la arquitectura que se ponen a prueba: validez de los episodios, utilidad de la formulación de aprendizaje, evaluación de las políticas obtenidas y extensión a varias instancias simultáneas.

6.1. Evolución de la aplicación de entrenamiento

El Cuadro 6.1 resume las ejecuciones que aportan evidencia relevante para la validación. No constituye una búsqueda exhaustiva de hiperparámetros ni un registro cronológico de desarrollo: cada grupo de experimentos se conserva porque permite identificar una condición necesaria para interpretar el entrenamiento o porque muestra el efecto de una modificación en la tarea aplicada.

Resultados de la aplicación y validación de la arquitectura

Experimento	Propósito	Evidencia observada	Aportación a la validación
Exp001	Configuración inicial de PPO.	Episodios de éxito en un paso y métricas insuficientes.	El objetivo estaba demasiado cerca del estado inicial; todavía no se representaba una tarea de navegación útil.
Exp002	Objetivo desplazado y prueba corta de monitorización.	Primeros episodios útiles, seguidos de bucles de salida de límites.	El pipeline generaba artefactos, pero el reinicio no garantizaba un inicio seguro tras OOB ¹ .
Exp003	Introducir <code>fixed_start_pose</code> .	Entrenamiento corto completado con monitor, TensorBoard y <i>checkpoints</i> .	La condición inicial fija eliminó el bucle inmediato de OOB en una prueba acotada.
Exp004–Exp008a	Estabilización incremental del reinicio.	Bloqueos de <code>go_to</code> , <i>timeouts</i> de reinicio y comandos aceptados sin movimiento.	Se localizó el cuello de botella en la interacción entre reinicio, plataforma de control y simulador.
Exp009	Primera ejecución de PPO sobre el ciclo de reinicio validado.	5000 pasos y 126 episodios sin fallos de reinicio.	Se obtuvo una base de episodios válida; el aprendizaje aún no podía interpretarse como resultado de la tarea.
Exp010	Recompensa mínima, randomización completa e hiperparámetros revisados.	12 852 pasos y 427 episodios antes del fallo; 359 OOB, 65 por baja altitud y 3 éxitos.	Las fronteras no observables y el coste del <i>teleport</i> dominaron la experiencia; el simulador se bloqueó tras cientos de reinicios de servicio.
Exp011	Área segura de muestreo y límite de distancia inicio-objetivo.	21 347 pasos y 637 episodios; 512 OOB, 121 por baja altitud y 3 éxitos.	La saturación de observaciones se corrigió, pero una política sin entrenar seguía derivando hacia fronteras no observables.
Exp012	Protección de frontera, corredor vertical ampliado, reanudación y supervisor.	704 episodios; picos del 24 % de éxito sobre un simulador recién iniciado.	La protección horizontal redujo las salidas laterales, pero la inercia vertical y la degradación acumulativa seguían afectando a la ejecución.
Exp013	Protecciones predictivas y empuje vertical con autoridad completa.	Entrenamiento de 50 000 pasos sin fallos; 0 <i>teleports</i> en los primeros 24 000 pasos.	Los episodios pudieron ejecutarse sin invalidaciones físicas; la tasa de éxito se mantuvo en torno al 11 %, señalando la dificultad de la tarea.
Exp014	Primera etapa del curriculum, con objetivos a $\leq 2,5$ m.	27 % de éxito global, con picos del 50 %.	La reducción inicial de distancia proporcionó una señal de éxito suficiente para iniciar el aprendizaje.
Exp015	Segunda etapa, con objetivos a $\leq 3,5$ m y <i>warm start</i> .	65 % de éxito global y picos del 90 %.	La política mantuvo el comportamiento aprendido al ampliar el rango de objetivos.
Exp016	Tercera etapa, con el rango completo de 5 m.	91 % global; 99 % (158/160) en el tramo final sobre un simulador recién iniciado.	La aplicación produjo aprendizaje útil en el rango completo de la tarea.
Exp017	Primera extensión vectorizada con dos drones.	1184 de 1189 episodios OOB, con terminaciones a los 2 pasos.	Se identificó la retención de la última referencia de velocidad durante reinicios bloqueantes como causa de la deriva entre instancias.
Exp018	Ejecución vectorizada con vigilancia de comandos inactivos.	96 % de éxito global en 1272 episodios; 100 % (244/244) en el tramo final.	La aplicación se ejecutó con dos instancias y mantuvo episodios interpretables durante el entrenamiento.

Cuadro 6.1: Ejecuciones relevantes de la aplicación de entrenamiento y evidencia aportada para validar la arquitectura.

La Figura 6.1 sintetiza la evolución de la tasa de éxito. Las ejecuciones Exp010–Exp013 muestran que disponer de un proceso de entrenamiento activo no basta para extraer conclusiones sobre PPO. Antes fue necesario reducir los reinicios costosos y asegurar que las transiciones fuesen físicamente válidas. El incremento posterior coincide con el curriculum de distancia (Exp014–Exp016) y se conserva al extender la aplicación a dos drones en Exp018.

¹OOB corresponde a *out of bounds*, es decir, una terminación por salida de los límites físicos definidos para el episodio.

6.2. Evidencia de aprendizaje mediante curriculum de distancia

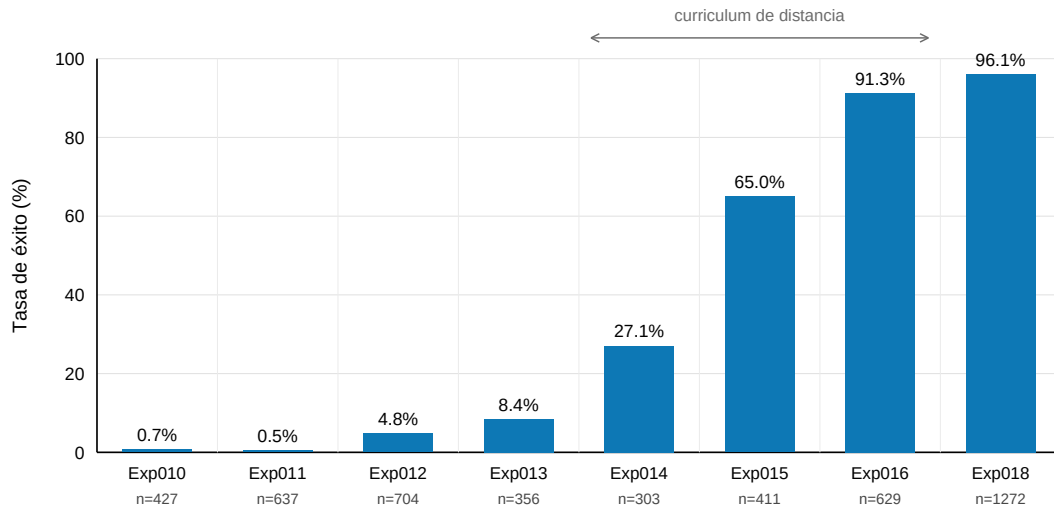


Figura 6.1: Tasa de éxito por experimento durante la campaña principal. Cada barra agrega los episodios registrados por el monitor de la ejecución; n indica el número de episodios.

6.2. Evidencia de aprendizaje mediante curriculum de distancia

Los experimentos Exp010–Exp013 indican que la recompensa mínima $-d_{\text{norm}}$ ofrece poca señal de éxito cuando el objetivo puede situarse a 5 m de distancia. Una vez eliminadas las causas de invalidez física, Exp013 completó 50 000 pasos sin fallos de reinicio ni *teleports* durante sus primeros 24 000 pasos, pero su tasa de éxito permaneció en torno al 11 %. Esto permite interpretar el estancamiento como una característica de la tarea planteada y no como un efecto inmediato del ciclo de reinicio o del canal de acción.

El curriculum de distancia se aplicó en tres etapas con *warm start*. La primera limita la separación inicio–objetivo a 2,5 m; las dos siguientes la amplían a 3,5 m y 5,0 m reutilizando el modelo final de la etapa precedente. La separación mínima de 1,0 m se mantuvo en todas ellas para que los objetivos cercanos permaneciesen en la distribución de episodios.

Resultados de la aplicación y validación de la arquitectura

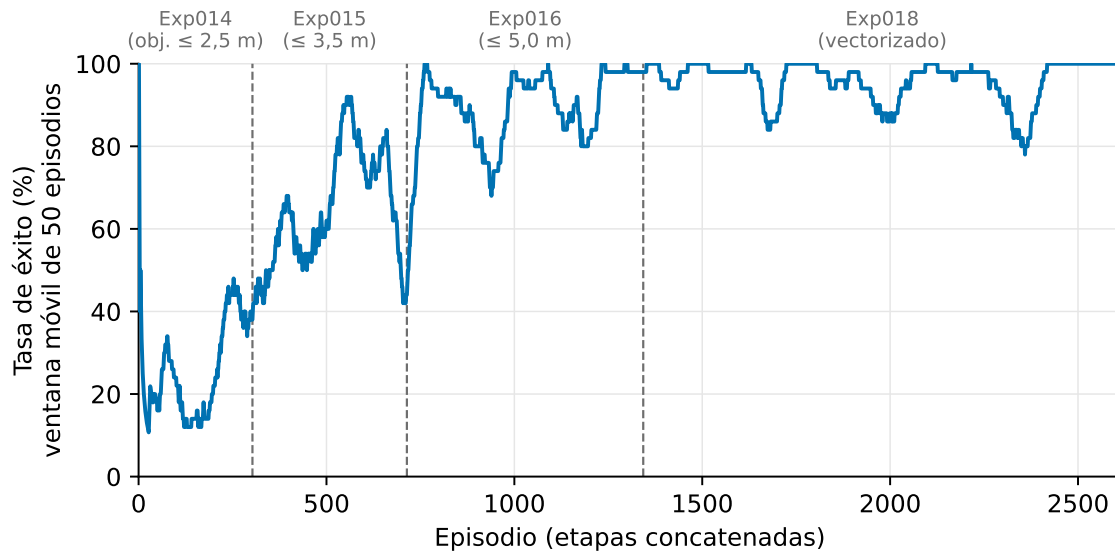


Figura 6.2: Tasa de éxito en ventana móvil de 50 episodios a lo largo de las tres etapas del currículum y del entrenamiento vectorizado posterior. Las líneas discontinuas marcan los cambios de etapa, en los que se amplía el rango de distancias muestreado.

La Figura 6.2 muestra descensos transitorios al ampliar el rango de distancias y recuperaciones posteriores de la tasa de éxito. En las condiciones evaluadas, este patrón indica que la política conserva parte del comportamiento adquirido al aumentar la dificultad de la tarea. La recompensa media por episodio no resulta suficiente para describir esa mejora de comportamiento, pues la penalización continua $-d_{\text{norm}}$ hace que los episodios largos y exitosos acumulen más coste que algunos episodios cortos que finalizan por fallo. Por ello, la interpretación se apoya principalmente en la tasa de éxito, la distancia final y la composición de las terminaciones.

6.3. Evaluación de las políticas PPO

La evaluación final utiliza 100 episodios generados con semillas fijas (1000–1099) sobre un simulador recién iniciado. Se analizan dos políticas: la obtenida al final de Exp016 y la resultante de la extensión vectorizada de Exp018. Cada una se ejecuta en modo determinista y estocástico. Esta distinción permite valorar si el comportamiento se mantiene al aplicar la media de la distribución aprendida o al muestrear acciones de dicha distribución.

Política evaluada	Éxito	Dist. final (m)	Pasos (med.)	Tiempo (med.)	Error yaw (rad)
PPO Exp016 determinista	78/100	0,43	30	6,0 s	0,58
PPO Exp016 estocástico	85/100	0,40	37	7,4 s	0,49
PPO Exp018 determinista	100/100	0,32	32	6,4 s	0,44
PPO Exp018 estocástico	89/100	0,36	31	6,2 s	0,49

Cuadro 6.2: Evaluación de las políticas PPO en 100 episodios con semillas fijas. Pasos y tiempo corresponden a la mediana de los episodios con éxito; el error de guiñada es la media absoluta en el instante terminal.

El Cuadro 6.2 muestra una mejora entre las políticas obtenidas tras Exp016 y Exp018.

En modo determinista, la política de Exp016 alcanzó 78 de 100 episodios y la de Exp018 completó los 100. Este resultado no pretende caracterizar PPO de forma general, sino aportar evidencia de que la arquitectura permite entrenar y evaluar una política sobre episodios reproducibles de la tarea definida.

La Figura 6.3 permite interpretar los 22 fallos de la evaluación determinista de Exp016. El histograma incluye los 100 episodios: las barras azules corresponden a los 78 éxitos, cuya distancia final queda por debajo del umbral de 0,4 m, y las barras naranjas corresponden a los 22 episodios que agotaron el límite de pasos sin alcanzar ese umbral. Por tanto, los fallos no se deben a una terminación de seguridad, sino a que la política no completó la aproximación final al objetivo. Algunos episodios fallidos quedan cerca de la región de éxito, mientras que otros terminan a mayor distancia. En la evaluación determinista de Exp018, los 100 episodios alcanzan el umbral y no aparece este patrón de truncamiento.

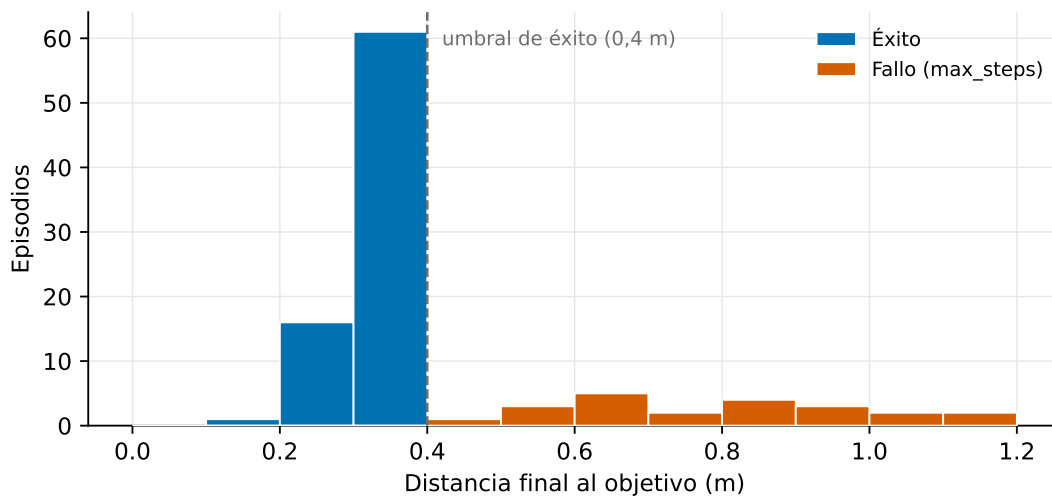


Figura 6.3: Distribución de la distancia final al objetivo en los 100 episodios de la evaluación determinista de Exp016. Las barras azules representan los éxitos y las naranjas los episodios truncados por alcanzar el límite de pasos. La línea discontinua marca el umbral de éxito de 0,4 m: los fallos quedan a su derecha, sin terminaciones de seguridad.

El modo de evaluación también influye en las tasas observadas. En Exp016, el modo estocástico alcanza un 85 % frente al 78 % del modo determinista; en Exp018 sucede lo contrario, con un 89 % frente al 100 %. Este contraste indica que el muestreo de acciones introduce variabilidad en el comportamiento de la política. Por esta razón, el modo determinista se utiliza como referencia principal para comparar ambos modelos, mientras que el modo estocástico aporta una comprobación complementaria. El error de guiñada terminal, situado entre 0,44 y 0,58 rad, es coherente con la formulación de la recompensa donde la orientación modula la bonificación de éxito, pero no se optimiza como un objetivo de seguimiento de trayectoria independiente.

6.4. Extensión a entrenamiento vectorizado

La vectorización se incorpora como una validación adicional de la infraestructura, no como una búsqueda de la política de mayor rendimiento. Esta sección persigue dos objetivos: comprobar que la arquitectura admite un entrenamiento PPO con varias

instancias en paralelo y observar si la política continúa mejorando durante esa extensión del entrenamiento. Por tanto, el resultado se interpreta como evidencia de integración de la arquitectura y de continuidad del aprendizaje, no como una comparación de configuraciones de PPO.

La evaluación se limita a dos drones, `drone0` y `drone1`. Esta elección responde a los recursos disponibles en el equipo de experimentación: `SubprocVecEnv` crea un proceso de trabajo por instancia del entorno y cada dron requiere además su propia integración con `Aerostack2` dentro de un simulador que avanza en tiempo real. El objetivo no es determinar el número máximo de vehículos que puede soportar la plataforma, sino demostrar la viabilidad de la ejecución vectorizada en una configuración acotada y representativa. Con dos instancias, cada paso vectorizado tarda 0,20 s, igual que un paso individual, y entrega dos transiciones; el caudal efectivo de muestras se duplica respecto a la ejecución monodron. Una vectorización secuencial no aportaría esta ventaja, pues N entornos ejecutados uno tras otro requerirían $N \times 0,2$ s para obtener N muestras.

Exp017 puso de manifiesto un riesgo propio de esta integración. El primer entrenamiento con dos drones acumuló 1184 terminaciones OOB de 1189 episodios, con una mediana de dos pasos y desplazamientos de hasta 4 m entre pasos consecutivos. El análisis mostró que `Aerostack2` mantiene la última referencia de velocidad si deja de recibir comandos. Cuando una instancia ejecuta un reinicio de 10–25 s, el paso vectorizado queda bloqueado y la otra instancia conserva esa consigna. En una prueba de aislamiento, 20 s sin comandos produjeron una deriva de 8,05 m. Este resultado no invalida la vectorización, sino que identifica una condición de seguridad que la arquitectura debe gestionar para usarla.

La capa de vigilancia de comandos inactivos descrita en el Capítulo 5 publica una referencia de velocidad nula cuando una instancia permanece más de 0,5 s sin nuevas órdenes. Esta medida redujo la deriva observada de 8,05 m a 0,27 m. Con ella, Exp018 prolongó durante 50 000 pasos el modelo final de Exp016 usando las dos instancias vectorizadas; obtuvo un 96 % de éxito global y un 100 % (244/244) en su tramo final. En consecuencia, Exp018 aporta dos evidencias complementarias: la arquitectura permite entrenar con dos procesos y dos drones coordinados sobre un simulador compartido, y la política siguió mejorando durante esta continuación vectorizada. Estos resultados no permiten afirmar que dos instancias constituyan un límite de escalabilidad ni que la vectorización, por sí sola, sea la causa de la mejora de la política.

6.5. Hallazgos de integración en el simulador

La aplicación de entrenamiento permitió identificar dos comportamientos del fork del simulador que condicionan la interpretación de sus resultados. El primero es la degradación acumulativa asociada al servicio de reinicio. Exp010 y Exp011 terminaron con el canal de comandos bloqueado tras 426 y 634 *teleports*, respectivamente. En Exp012 se observó que el efecto era gradual, la tasa de éxito de la misma política descendió del 9 % al 1 % tras cientos de *teleports* y alcanzó el 24 % tras reiniciar el proceso del simulador.

La mitigación aplicada combina varios elementos descritos a lo largo de la memoria como las protecciones de frontera, que reducen las terminaciones que exigen *tele-*

port; un supervisor que reinicia el conjunto de servicios y reanuda desde el último *checkpoint*; y el reciclado preventivo del proceso cada 2,5 horas. Estas medidas hacen viable la campaña de entrenamiento, pero no corrigen la causa raíz en el simulador. El segundo hallazgo es la retención de la última referencia de velocidad ya descrita para Exp017. Aunque resulta razonable en teleoperación, exige una protección adicional cuando el entrenamiento paraleliza instancias sobre un simulador compartido.

6.6. Limitaciones

Los resultados deben interpretarse dentro de las condiciones en que se obtuvieron. En primer lugar, las protecciones de frontera modifican la dinámica efectivamente experimentada por la política, las terminaciones de seguridad se mantienen como última barrera, pero son menos frecuentes que sin esta protección. En segundo lugar, la degradación del simulador se ha mitigado operativamente, no eliminado en su origen, por lo que las campañas largas siguen dependiendo de reinicios preventivos y supervisión.

Además, la evaluación se limita a una única tarea de navegación local, a las configuraciones y semillas indicadas, a dos instancias vectorizadas y a un equipo de ejecución concreto. El simulador emplea una dinámica simplificada y no se ha realizado transferencia a una plataforma física. Por tanto, los resultados validan la utilidad de la arquitectura para esta aplicación de control de velocidad y sus condiciones de integración, pero no permiten generalizar el comportamiento observado a otras tareas, simuladores o vehículos reales.

Capítulo 7

Conclusiones y líneas futuras

Este capítulo revisa los objetivos definidos en el Capítulo 1 a partir de la evidencia de diseño, implementación y validación presentada en los capítulos anteriores. La contribución principal del trabajo es la construcción de una arquitectura software reproducible para el entrenamiento mediante aprendizaje por refuerzo de robots aéreos sobre un simulador integrado con Aerostack2 y ROS 2. El entrenamiento de políticas PPO se utiliza como una aplicación de validación de la arquitectura.

7.1. Conclusiones

El objetivo general de desarrollar una arquitectura software para el entrenamiento mediante aprendizaje por refuerzo de robots aéreos autónomos en un simulador integrado con un *stack* robótico se ha alcanzado en el alcance definido. La arquitectura implementa el contrato de interacción necesario para PPO, con observaciones relativas, acciones continuas de velocidad y guiñada, recompensa, condiciones de terminación, reinicio y monitorización por episodio. Su integración con Aerostack2 permite que cada acción se traduzca en una referencia para el UAV simulado y que la transición devuelta al agente incorpore el estado físico resultante.

El primer objetivo específico consistía en analizar el problema de navegación en simulación y definir los requisitos funcionales de la arquitectura y de su interfaz de aprendizaje por refuerzo. Este objetivo se materializó en una tarea de navegación local hacia objetivos relativos, con distancias iniciales y objetivos variables. La formulación hace explícitos los espacios de observación y acción, el umbral de éxito de 0,4 m, las condiciones de seguridad y el límite temporal. El análisis también permitió identificar un requisito que no era puramente algorítmico y es que para que el entrenamiento sea interpretable, los episodios deben comenzar en estados verificables y los comandos aceptados deben producir desplazamiento físico medible.

El segundo objetivo específico era diseñar e implementar una arquitectura que integre el simulador dinámico y el *middleware* robótico para ejecutar el ciclo de interacción de forma reproducible. Se desarrolló una capa Gymnasium sobre Aerostack2, configuraciones declarativas de entrenamiento, monitorización por episodio, mecanismos de supervisión y comprobaciones de encapsulamiento e integración. La principal conclusión es que la interfaz de aprendizaje no basta por sí sola ya que la fiabilidad del reinicio y la accionabilidad del canal de comandos condicionan la validez

de todos los datos recogidos. La solución incorpora verificaciones del estado posterior al reinicio, protección predictiva de fronteras y trazas que permiten distinguir un fallo de la política de un fallo de la infraestructura.

El tercer objetivo específico consistía en validar la arquitectura mediante políticas de navegación con estados iniciales y objetivos variables, prestando atención al reinicio, la accionabilidad y la reproducibilidad. Esta validación se realizó de forma progresiva. Exp013 completó 50 000 pasos sin fallos de reinicio y sin *teleports* en sus primeros 24 000 pasos, lo que permitió separar los problemas de integración de los problemas de aprendizaje. Sobre esa base, el curriculum de distancia elevó la tasa de éxito global del 27 % en Exp014 al 65 % en Exp015 y al 91 % en Exp016, ya sobre el rango completo de 5 m. La evaluación posterior de la política de Exp018 en 100 episodios con semillas fijas alcanzó 100 éxitos en modo determinista, con una distancia final media de 0,32 m. Estos resultados validan que la arquitectura permite entrenar y evaluar una política en la tarea propuesta, sin pretender generalizar el comportamiento de PPO a otras configuraciones o problemas de control.

La extensión vectorizada aporta una validación adicional de la infraestructura. La configuración se limitó a dos drones y dos procesos de trabajo por las restricciones de recursos del equipo de experimentación y del simulador en tiempo real. En estas condiciones, dos instancias mediante `SubprocVecEnv` mantuvieron un tiempo de paso de 0,20 s y duplicaron el caudal de muestras respecto a la ejecución monodron. El fallo inicial de Exp017 reveló que la retención de la última referencia de velocidad durante un reinicio bloqueante podía producir 8,05 m de deriva en 20 s. El mecanismo de vigilancia de comandos inactivos redujo esa deriva a 0,27 m y permitió completar Exp018 con dos drones. La política continuó mejorando durante esta ejecución, hasta un 96 % de éxito global y un 100 % (244/244) en su tramo final. La conclusión no es que dos instancias sean un límite de escalabilidad ni que la vectorización sea la única causa de la mejora, sino que la arquitectura puede sostener entrenamiento vectorizado en una configuración acotada y diagnosticable.

Los resultados negativos también constituyen una aportación relevante. Las ejecuciones Exp010, Exp011, Exp012 y Exp017 permitieron identificar tres límites de integración: la degradación acumulativa asociada a reinicios de servicio, la insuficiencia de las protecciones reactivas frente a la inercia vertical y la persistencia de referencias de velocidad cuando una instancia deja de recibir comandos. Documentar estos comportamientos y las mitigaciones aplicadas evita interpretar como aprendizaje resultados que, en realidad, dependen de la plataforma de simulación.

El código queda organizado en los repositorios públicos descritos en el Capítulo 4, junto con la configuración de las ejecuciones, las comprobaciones automatizadas y los mecanismos de monitorización. De este modo, el resultado del trabajo no se limita a una política concreta, sino a una base técnica reutilizable para formular y validar nuevas tareas de aprendizaje por refuerzo sobre robots aéreos simulados.

7.2. Líneas futuras

Una primera línea de trabajo consiste en corregir la degradación del simulador ante reinicios de servicio repetidos o, alternativamente, incorporar una interfaz directa con la dinámica del simulador que permita controlar explícitamente el avance temporal. Un simulador pausable reduciría el coste de los reinicios y evitaría que el tiempo físico

Conclusiones y líneas futuras

continuase avanzando durante operaciones de reinicio bloqueantes en el simulador. Esta mejora atacaría las limitaciones observadas en las campañas largas y facilitaría aumentar el número de instancias sin depender de la coordinación en tiempo real de ROS 2.

La segunda línea es estudiar el escalado de la vectorización. El resultado con dos drones demuestra la viabilidad de la integración, pero no determina cómo se comporta la arquitectura con más procesos, vehículos o recursos de cómputo. Sería necesario medir de forma sistemática el uso de CPU y memoria, el tiempo de paso, la frecuencia de reinicios y la estabilidad de la simulación para identificar un compromiso reproducible entre paralelismo y coste computacional.

En relación con la tarea de aprendizaje, resulta de interés extender la formulación a objetivos dinámicos, trayectorias con puntos intermedios, perturbaciones y observaciones de mayor complejidad. También puede revisarse la recompensa terminal para otorgar mayor importancia a la orientación final, que en la formulación actual solo modula la bonificación de éxito. Estas extensiones permitirían comprobar si la arquitectura conserva su utilidad cuando la tarea deja de ser una navegación local hacia un único objetivo.

Por último, la transferencia a una plataforma física requeriría incorporar modelos de sensores, perturbaciones y retardos más representativos, así como un protocolo de seguridad para ejecutar políticas entrenadas en simulación. Antes de esa transferencia conviene emplear la infraestructura construida para evaluar la robustez de las políticas ante variaciones de la dinámica y de la estimación de estado. Este paso delimita una continuación realista del trabajo sin atribuir al entorno simulado garantías que solo pueden comprobarse sobre un vehículo real.

Bibliografía

- AltexSoft (2021). *Reinforcement Learning Explained: Overview, Comparisons and Applications in Business*. URL: <https://www.altexsoft.com/blog/reinforcement-learning-explained-overview-comparisons-and-applications-in-business/> (visitado 28-06-2026).
- Anthropic (2026). *Claude Code*. URL: <https://claude.com/claude-code> (visitado 06-07-2026).
- Azar, Ahmad Taher, Anis Koubaa, Nada Ali Mohamed, Habiba A. Ibrahim, Zahra Fathy Ibrahim, Muhammad Kazim, Adel Ammar, Bilel Benjdira, Alaa M. Khamis, Ibrahim A. Hameed y Gabriella Casalino (2021). «Drone Deep Reinforcement Learning: A Review». En: *Electronics* 10.9, pág. 999. DOI: 10.3390/electronics10090999. URL: <https://doi.org/10.3390/electronics10090999>.
- Bengio, Yoshua, Jérôme Louradour, Ronan Collobert y Jason Weston (2009). «Curriculum Learning». En: *Proceedings of the 26th International Conference on Machine Learning (ICML)*, págs. 41-48.
- Chan, Jun Hoong, Kai Liu, Yu Chen, A. S. M. Sharifuzzaman Sagar y Yong-Guk Kim (2024). «Reinforcement Learning-based Drone Simulators: Survey, Practice, and Challenge». En: *Artificial Intelligence Review* 57.10, pág. 281. DOI: 10.1007/s10462-024-10933-w. URL: <https://doi.org/10.1007/s10462-024-10933-w>.
- Chollet, Gabriel (2021). *What is ROS*. URL: https://medium.com/@Gabriel_Chollet/what-is-ros-c38493fe3eca (visitado 28-06-2026).
- Eschmann, Jonas, Dario Albani y Giuseppe Loianno (2024). «Learning to Fly in Seconds». En: *IEEE Robotics and Automation Letters* 9.7, págs. 6336-6343. DOI: 10.1109/LRA.2024.3396025. URL: <https://doi.org/10.1109/LRA.2024.3396025>.
- Farama Foundation (2026). *Gymnasium Documentation: Env API*. URL: <https://gymnasium.farama.org/api/env/> (visitado 25-06-2026).
- Fernández-Cortizas, Miguel, Martín Molina, Pedro Arias-Perez, Rafael Perez-Segui, David Perez-Saura y Pascual Campoy (2023). «Aerostack2: A Software Framework for Developing Multi-robot Aerial Systems». En: *arXiv preprint arXiv:2303.18237*. DOI: 10.48550/arXiv.2303.18237. URL: <https://arxiv.org/abs/2303.18237>.
- Kaufmann, Elia, Leonard Bauersfeld y Davide Scaramuzza (2022). «A Benchmark Comparison of Learned Control Policies for Agile Quadrotor Flight». En: *2022 International Conference on Robotics and Automation (ICRA)*, págs. 10504-10510. DOI: 10.1109/ICRA46639.2022.9811564. URL: <https://doi.org/10.1109/ICRA46639.2022.9811564>.
- Kaufmann, Elia, Antonio Loquercio, Rene Ranftl, Matthias Müller, Vladlen Koltun y Davide Scaramuzza (2020). «Deep Drone Acrobatics». En: *Proceedings of Robotics: Science and Systems*. Corvallis, Oregon, USA. DOI: 10.15607/RSS.2020.XVI.040. URL: <https://www.roboticsproceedings.org/rss16/p040.html>.

- Krishnan, Srivatsan, Behzad Boroujerdian, William Fu, Aleksandra Faust y Vijay Janapa Reddi (2021). «Air Learning: a Deep Reinforcement Learning Gym for Autonomous Aerial Robot Visual Navigation». En: *Machine Learning* 110, págs. 2501-2540. DOI: 10.1007/s10994-021-06006-6. URL: <https://doi.org/10.1007/s10994-021-06006-6>.
- Molina, Martín (2025a). «Actor Critic Policy Optimization». Material docente de la asignatura Artificial Neural Networks and Deep Learning, Máster Universitario en Inteligencia Artificial, Universidad Politécnica de Madrid.
- (2025b). «Introduction to Reinforcement Learning». Material docente de la asignatura Artificial Neural Networks and Deep Learning, Máster Universitario en Inteligencia Artificial, Universidad Politécnica de Madrid.
- (2025c). «Proximal Policy Optimization». Material docente de la asignatura Artificial Neural Networks and Deep Learning, Máster Universitario en Inteligencia Artificial, Universidad Politécnica de Madrid.
- Objectways (2025). *Deep Learning Vs Reinforcement Learning: Key Differences*. URL: <https://objectways.com/blog/deep-learning-vs-reinforcement-learning-key-differences/> (visitado 28-06-2026).
- Open Robotics (2026). *ROS 2 Humble Documentation: Basic Concepts*. URL: <https://docs.ros.org/en/humble/Concepts.html> (visitado 25-06-2026).
- OpenAI (2026a). *ChatGPT*. URL: <https://chatgpt.com/> (visitado 02-07-2026).
- (2026b). *OpenAI Codex*. URL: <https://openai.com/codex/> (visitado 02-07-2026).
- Panerati, Jacopo, Hehui Zheng, Siqi Zhou, James Xu, Amanda Prorok y Angela P. Schoellig (2021). «Learning to Fly — a Gym Environment with PyBullet Physics for Reinforcement Learning of Multi-agent Quadcopter Control». En: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, págs. 7512-7519. DOI: 10.1109/IROS51168.2021.9635857. URL: <https://doi.org/10.1109/IROS51168.2021.9635857>.
- Pi, Chen-Huan, Yi-Wei Dai, Kai-Chun Hu y Stone Cheng (2021). «General Purpose Low-Level Reinforcement Learning Control for Multi-Axis Rotor Aerial Vehicles». En: *Sensors* 21.13, pág. 4560. DOI: 10.3390/s21134560. URL: <https://doi.org/10.3390/s21134560>.
- ResearchGate (2024). *Proximal Policy Optimization (PPO) architecture*. URL: https://www.researchgate.net/figure/Proximal-Policy-Optimization-PPO-architecture_fig2_385920419 (visitado 28-06-2026).
- Sánchez-López, José Luis, Martín Molina, Hriday Bavle, Carlos Sampedro, Ramón Suárez Fernández, David Palacios, Antonio Díaz-Moreno y Pascual Campoy (2017). «A Multi-Layered Component-Based Approach for the Development of Aerial Robotic Systems: The Aerostack Framework». En: *Journal of Intelligent & Robotic Systems* 88.2-4, págs. 683-709. DOI: 10.1007/s10846-017-0551-4. URL: <https://doi.org/10.1007/s10846-017-0551-4>.
- Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford y Oleg Klimov (2017). «Proximal Policy Optimization Algorithms». En: *arXiv preprint arXiv:1707.06347*. DOI: 10.48550/arXiv.1707.06347. URL: <https://arxiv.org/abs/1707.06347>.
- Song, Yunlong, Selim Naji, Elia Kaufmann, Antonio Loquercio y Davide Scaramuzza (2021). «Flightmare: A Flexible Quadrotor Simulator». En: *Proceedings of the 5th Conference on Robot Learning*. Vol. 155. Proceedings of Machine Learning Research. PMLR, págs. 1147-1157. URL: <https://proceedings.mlr.press/v155/song21a.html>.

- Song, Yunlong, Angel Romero, Matthias Müller, Vladlen Koltun y Davide Scaramuzza (2023). «Reaching the Limit in Autonomous Racing: Optimal Control versus Reinforcement Learning». En: *Science Robotics* 8.82, eadg1462. DOI: 10.1126/scirobotics.adg1462. URL: <https://doi.org/10.1126/scirobotics.adg1462>.
- Stable-Baselines3 Contributors (2026). *Stable-Baselines3 Documentation: PPO*. URL: <https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html> (visitado 25-06-2026).
- Sutton, Richard S. y Andrew G. Barto (2018). *Reinforcement Learning: An Introduction*. 2.^a ed. MIT Press. URL: <http://incompleteideas.net/book/the-book-2nd.html>.
- Towers, Mark, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan y Omar G. Younis (2024). «Gymnasium: A Standard Interface for Reinforcement Learning Environments». En: *arXiv preprint arXiv:2407.17032*. DOI: 10.48550/arXiv.2407.17032. URL: <https://arxiv.org/abs/2407.17032>.
- Yoo, Jaehyun, Dohyun Jang, H. Jin Kim y Karl H. Johansson (2021). «Hybrid Reinforcement Learning Control for a Micro Quadrotor Flight». En: *IEEE Control Systems Letters* 5.2, págs. 505-510. DOI: 10.1109/LCSYS.2020.3001663. URL: <https://doi.org/10.1109/LCSYS.2020.3001663>.

Apéndice A

Anexos

Los anexos recogen información complementaria para facilitar la reproducción y la inspección de los experimentos sin interrumpir la discusión principal. No se plantean como un registro exhaustivo de todas las ejecuciones, sino como una guía de lectura del repositorio: se resumen las configuraciones más relevantes, se indican los puntos de entrada de ejecución y se muestra un ejemplo de visualización de métricas en TensorBoard. A partir de esas referencias, cualquier ejecución puede consultarse directamente en los directorios de configuración, scripts y artefactos experimentales del repositorio.

A.1. Configuraciones representativas

Las configuraciones Exp007, Exp008 y Exp008a representan la evolución desde un entrenamiento largo bloqueado por reset hacia ejecuciones de curriculum con mayor instrumentación. Las configuraciones Exp009–Exp012 corresponden a la fase en la que el reinicio del simulador ya está certificado y el foco se desplaza hacia la validez de la recompensa, la randomización de episodios y la reducción de salidas de límites. Las configuraciones Exp013–Exp018 forman la campaña final: guards predictivos, curriculum de distancia en tres etapas y entrenamiento vectorizado con watchdog.

A.2. Comandos representativos

Config.	Objetivo	Duración	Rasgo principal
Exp007	Target desplazado es- table.	20000 pasos.	Gate de reset tras eliminar fallback bloqueante.
Exp008	Target de 2 m.	20000 pasos.	Diagnóstico reforzado de re- set y servicio del simulador.
Exp008a	Target de 0,5 m.	5000 pasos.	Curriculum inicial, yaw diagnóstico y reset_max_vel.
Exp009	Stack de reset certifi- cado.	5000 pasos.	Primera ejecución PPO com- pleta sobre el reinicio certifi- cado.
Exp010	Reward mínima y ran- domización completa.	50000 pasos pla- nificados.	Recompensa alineada con el tutor e hiperparámetros PPO revisados.
Exp011	Área segura y anti- saturación.	50000 pasos pla- nificados.	Margen de muestreo y límite de distancia inicio-objetivo.
Exp012	Guards de límites y reanudación.	50000 pasos pla- nificados.	Guard de acción, corredor vertical ampliado y supervi- sor con <i>resume</i> .
Exp013	Guards predictivos.	50000 pasos.	Disparo por posición + velo- cidad $\times 1$ s y empuje vertical con autoridad completa.
Exp014	Curriculum etapa 1.	50000 pasos.	Objetivos a $\leq 2,5$ m; primer aprendizaje sostenido.
Exp015	Curriculum etapa 2.	100000 pasos acumulados.	Objetivos a $\leq 3,5$ m con <i>warm start</i> de Exp014.
Exp016	Curriculum etapa 3.	150000 pasos acumulados.	Rango completo de 5 m; va- lidación en la etapa final del curriculum.
Exp017	Primer vectorizado (2 drones).	Detenido.	Colapso por deriva en <i>locks- tep</i> ; origen del diagnóstico del watchdog.
Exp018	Vectorizado con watchdog.	200000 pasos acumulados.	2 drones en paralelo, recicla- do preventivo y modelo final.

Cuadro A.1: Configuraciones representativas de la fase de estabilización.

A.2. Comandos representativos

La reproducción de una ejecución no depende de copiar literalmente un comando aislado, sino de combinar tres elementos del repositorio: el script que define el punto de entrada, el fichero YAML que fija la configuración del experimento y el directorio de salida donde quedan registrados los artefactos. Por este motivo, los comandos siguientes se presentan como patrones de uso y no como recetas cerradas. Antes de ejecutar pruebas con simulador conviene reiniciar AS2 para reducir el efecto de estados residuales del controlador o de la dinámica.

Listing A.1: Patrón general de entrenamiento PPO a partir de una configuración YAML.

```

1 python3 scripts/train_ppo.py \
2   --config configs/<configuracion_experimento>.yaml \
3   --num-envs <numero_entornos> \
4   --total-timesteps <pasos_entrenamiento>

```


La ejecución supervisada se utiliza en entrenamientos largos, especialmente cuando el simulador puede degradarse tras muchos reinicios. En este caso, el supervisor reinicia el stack de AS2, valida la disponibilidad del servicio de reinicio y permite reanudar desde un *checkpoint* o desde el modelo final de una etapa previa del curriculum.

Listing A.2: Patrón general de entrenamiento supervisado con reanudación.

```
1 RECYCLE_SECONDS=<segundos> \
2 DRONE_NAMESPACES=<drone0,drone1> \
3 INITIAL_RESUME=runs/ppo/<ejecucion_previa>/model_final.zip \
4 bash scripts/supervise_train.sh configs/<configuracion_experimento>.yaml
```

La evaluación formal permite ejecutar una política entrenada sobre episodios reproducibles. La configuración, el número de episodios y la semilla inicial se mantienen constantes para facilitar el análisis de los resultados.

Listing A.3: Patrón general de evaluación por semillas.

```
1 python3 scripts/evaluate_policy.py \
2 --config configs/<configuracion_experimento>.yaml \
3 --model runs/ppo/<ejecucion>/model_final.zip \
4 --episodes <n_episodios> --seed <semilla>
```

La Tabla A.2 resume dónde consultar cada tipo de información cuando se quiera revisar una ejecución concreta, más allá de los ejemplos incluidos en este anexo.

Elemento	Ruta principal	Uso dentro del análisis
Configuraciones	configs/	Define recompensa, límites, curriculum, hiperparámetros y parámetros de ejecución.
Scripts de ejecución	scripts/	Contiene los puntos de entrada para entrenamiento, evaluación, supervisión y validaciones acotadas.
Modelos y checkpoints	runs/ppo/<ejecucion>/	Permite recuperar el modelo final, reanudar entrenamientos y auditar artefactos generados.
Monitores de episodio	runs/ppo/<ejecucion>/mon	Fuente principal para tasas de éxito, duración, recompensa, distancias y causas de terminación.
TensorBoard	runs/ppo/<ejecucion>/ten	Fuente complementaria para inspeccionar escalares de PPO y métricas agregadas durante el entrenamiento.

Cuadro A.2: Rutas de consulta de configuraciones, scripts y artefactos experimentales en el repositorio.

A.3. Capturas de TensorBoard

TensorBoard se utiliza en este trabajo como herramienta de inspección, no como fuente única de evidencia. Las curvas permiten observar la evolución de escalares

A.4. Declaración de uso de herramientas de inteligencia artificial

registrados por Stable-Baselines3, como recompensa media, longitud media de episodio, tasa de éxito o métricas internas de PPO. Sin embargo, la interpretación de un experimento debe contrastarse siempre con los monitores de episodio y con las causas de terminación descritas en el Capítulo 6.

La Figura A.1 muestra un ejemplo de cómo se visualizan las métricas de *rollout* en TensorBoard para una ejecución concreta. No se incluyen capturas de todos los experimentos para evitar duplicar información ya disponible en el repositorio. Cuando se quiera revisar cualquier otra ejecución, basta con abrir el directorio de TensorBoard correspondiente al experimento deseado:

Listing A.4: Consulta de métricas TensorBoard para una ejecución concreta.

```
1 tensorboard --logdir runs/ppo/<ejecucion>/tensorboard
```

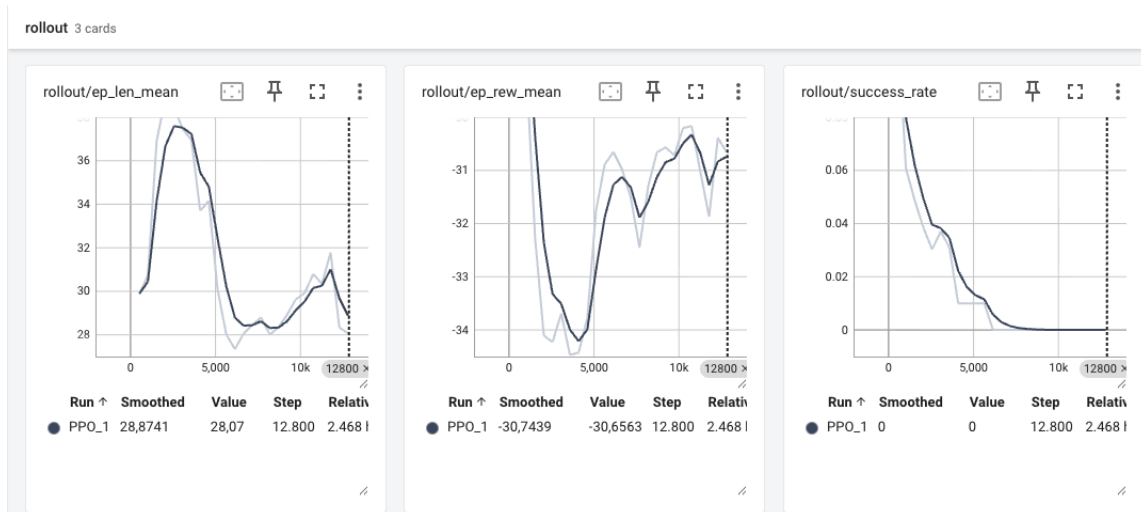


Figura A.1: Ejemplo de métricas de *rollout* de TensorBoard para Exp010. La captura ilustra el formato de consulta de los escalares registrados durante el entrenamiento; la interpretación del experimento se realiza junto con los monitores de episodio y la matriz de resultados.

A.4. Declaración de uso de herramientas de inteligencia artificial

Durante la realización del presente trabajo se han empleado herramientas de inteligencia artificial generativa como apoyo a tareas de programación, diagnóstico, organización de resultados y redacción de la memoria. En particular, se ha utilizado OpenAI Codex en el entorno de desarrollo para asistencia sobre código, inspección del repositorio, generación de propuestas de modificación, revisión de errores y edición de la memoria [OpenAI, 2026b]. También se ha utilizado ChatGPT como asistente conversacional para estructurar explicaciones, revisar la redacción y sintetizar criterios de presentación académica [OpenAI, 2026a]. Asimismo, se ha utilizado Claude Code como agente de desarrollo en la fase final del trabajo, con apoyo a la parte experimental, incluyendo la supervisión de entrenamientos, el análisis de registros y el diagnóstico de fallos de integración, así como a la implementación asistida de componentes y sus pruebas [Anthropic, 2026].

El uso de estas herramientas no sustituye la autoría ni la responsabilidad académica del trabajo. Las decisiones de diseño, la interpretación de resultados, la selección de experimentos y la aceptación final de cambios se han revisado a partir del repositorio, de las trazas de ejecución, de los ficheros de configuración y de los logs experimentales. Las respuestas generadas por IA se han tratado como propuestas de trabajo que debían ser verificadas, corregidas o descartadas antes de incorporarse al documento o al código.

Herramienta	Tareas asistidas	Validación aplicada
OpenAI Codex	Inspección del repositorio, edición de LaTeX, revisión de errores de compilación y apoyo a cambios de código.	Revisión manual de los diffs, compilación de LaTeX y contraste con el repositorio.
ChatGPT	Estructuración de explicaciones, síntesis de resultados y apoyo a redacción académica.	Revisión del texto, eliminación de afirmaciones no respaldadas y contraste con evidencias experimentales.
Claude Code	Apoyo a la parte experimental e implementación asistida de componentes y pruebas automatizadas.	Verificación de cifras contra los registros experimentales y revisión manual de los cambios.

Cuadro A.3: Resumen del uso de herramientas de inteligencia artificial generativa.

Los prompts más relevantes se orientaron a contrastar el estado del repositorio con el registro experimental, identificar diferencias entre indicadores de entrenamiento y de infraestructura, revisar incidencias de compilación de LaTeX y organizar la incorporación de figuras y tablas.

No se han utilizado las herramientas de inteligencia artificial como fuente bibliográfica primaria sobre aprendizaje por refuerzo, ROS 2, Aerostack2 o Stable-Baselines3. Las afirmaciones técnicas del cuerpo de la memoria se apoyan en referencias bibliográficas, documentación técnica, código implementado y evidencia experimental. Cuando se ha incorporado texto redactado con apoyo de IA, este se ha revisado y adaptado para mantener el tono académico, la precisión terminológica y la coherencia con los resultados disponibles.